

Pico Servo Driver

From Waveshare Wiki

Jump to: navigation, search

Overview

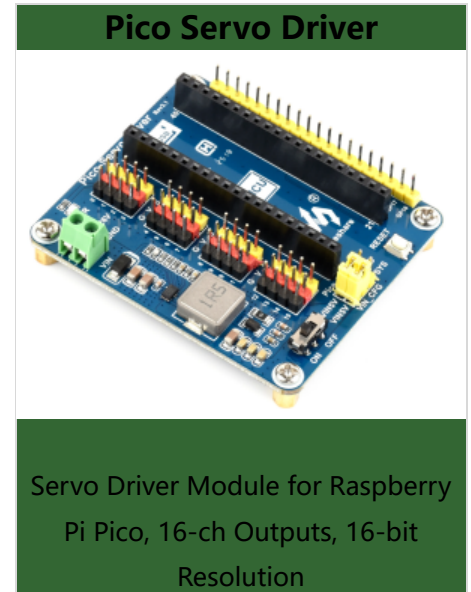
Servo Driver Module For Raspberry Pi Pico, 16-Channel Outputs, 16-Bit Resolution.

Features

- Standard Raspberry Pi Pico header, supports Raspberry Pi Pico series boards.
- Up to 16-channel servo/PWM outputs, 16-bit resolution for each channel.
- Integrates 5V regulator, up to 8A output current, allows battery power supply from the VIN terminal.
- Standard servo interface, supports commonly used servos such as SG90, MG90S, MG996R, etc.
- Adapting unused pins of Pico, easy expansion.
- Comes with online resources and manuals (including Raspberry Pi Pico C/C++ and MicroPython demo).

Specification

- Operating voltage: 5V (Pico) or 4.5~26V (VIN terminal)
- Logic voltage: 3.3V
- Servo voltage level: 5V
- Control interface: GPIO
- Mounting hole size: 3.0mm



- Dimensions: 65 × 56mm

Pinout

Channel 0~15, for concurrently connecting up to 16x servo

16x channels servo headers

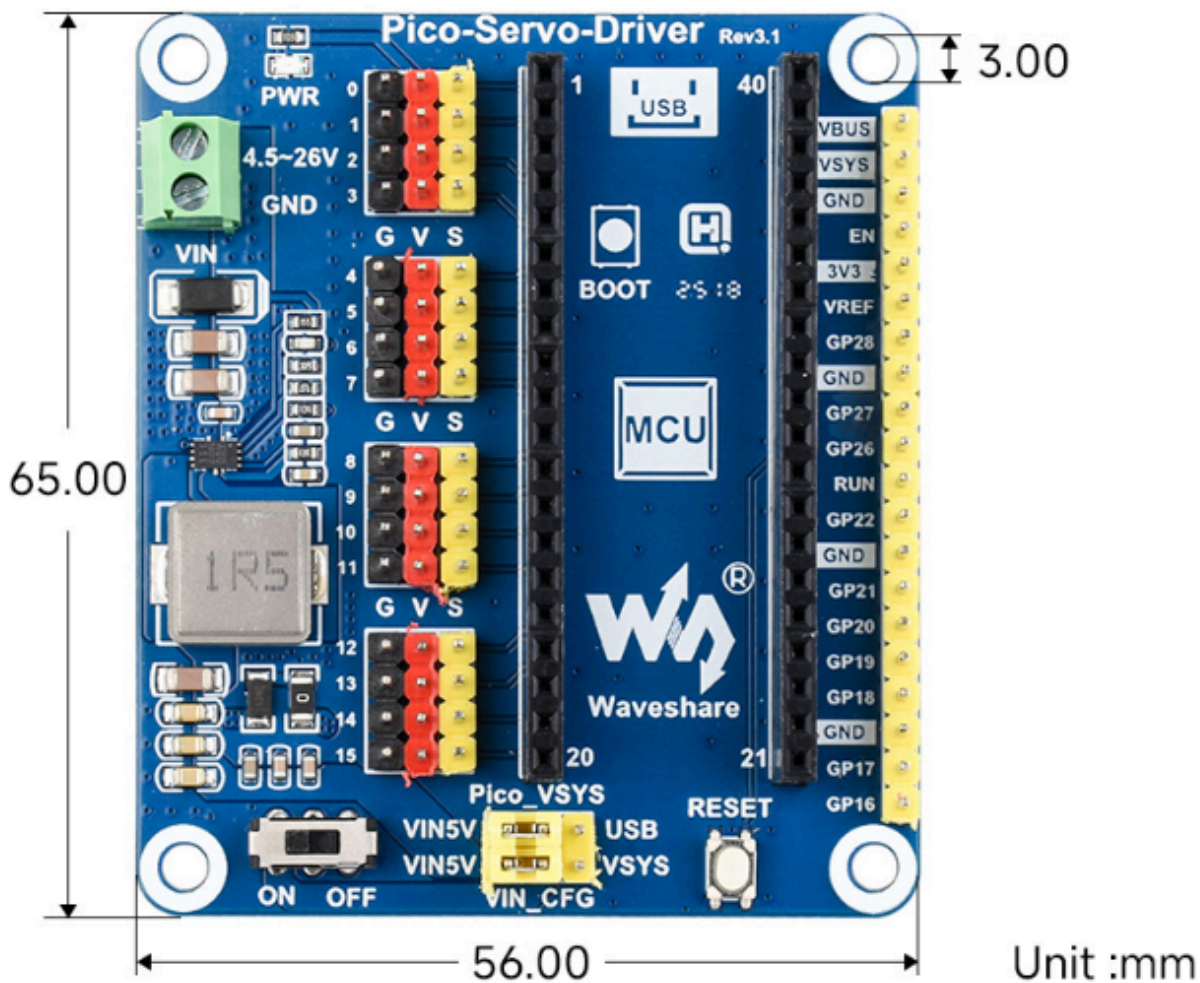
■ G pins: GND
■ V pins: 5V power
■ S pins: PWM signal

* please correctly connect the servo, it would not work if the connection were reversed.



(/wiki/File:700px-Pico-Servo-Driver-details-5.png)

Dimensions



(/wiki/File:Pico-

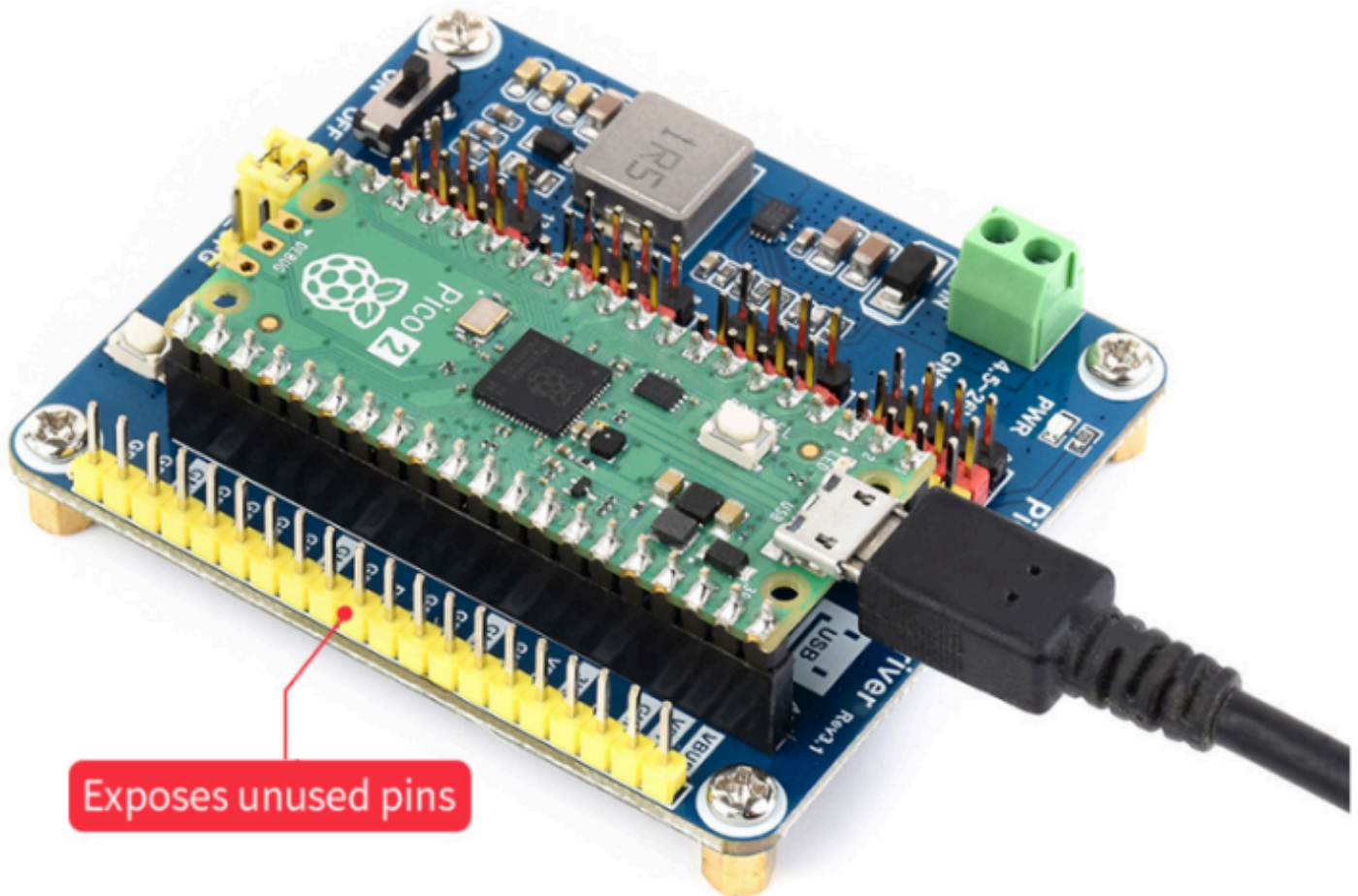
servo-driver-guide01.png)

User Guide

Hardware Connection

When connecting the Pico, please be careful not to reverse the corresponding direction. You can observe the end of the module with USB silkscreen and the end of Pico's USB interface to determine the direction (you can also judge the pin number of the motherboard on the module

and the pin number of Pico).



(/wiki/File:Pico_Servo_Driver_Guid02.png)

Demo Download

1. Download the demo from #Resource.
2. Open a terminal of Raspberry Pi and execute:

```
sudo apt-get install p7zip-full
cd ~
sudo wget https://files.waveshare.com/upload/3/31/Pico_Servo_Driver_Code.7z
7z Pico_Servo_Driver_Code.7z -o./Pico_Servo_Driver_Code
cd ~/Pico_Servo_Driver_Code
```

Raspberry Pi

c

- The following tutorials are operated on Raspberry Pi. As CMake has multi-platforms and can be ported, you also can successfully compile on a PC in different ways.

To compile, make sure you are in the c directory:

```
cd ~/Pico_Servo_Driver_Code/c/
```


Create and enter the "build" directory in the folder, and add the SDK:

where ../../pico-sdk is the directory of your SDK.

There is a "build" in our sample program, just enter it directly.

```
cd build
export PICO_SDK_PATH=../../pico-sdk
(Note: Be sure to write the path where your own SDK is located)
```

Execute cmake to automatically generate Makefile files

```
cmake..
```

Execute make to generate executable files, the first compilation time is relatively long.

```
make -j9
```

After the compilation is complete, the uf2 file will be generated.

Press and hold the button on the Pico board, connect the pico to the USB port of the Raspberry Pi through the Micro USB cable, and release the button. After connecting, the Raspberry Pi will automatically recognize a removable disk (RPI-RP2) and copy the main.uf2 file in the build folder to the recognized removable disk (RPI-RP2).

```
cp main.uf2 /media/pi/RPI-RP2/
```

python

Raspberry Pi Environment

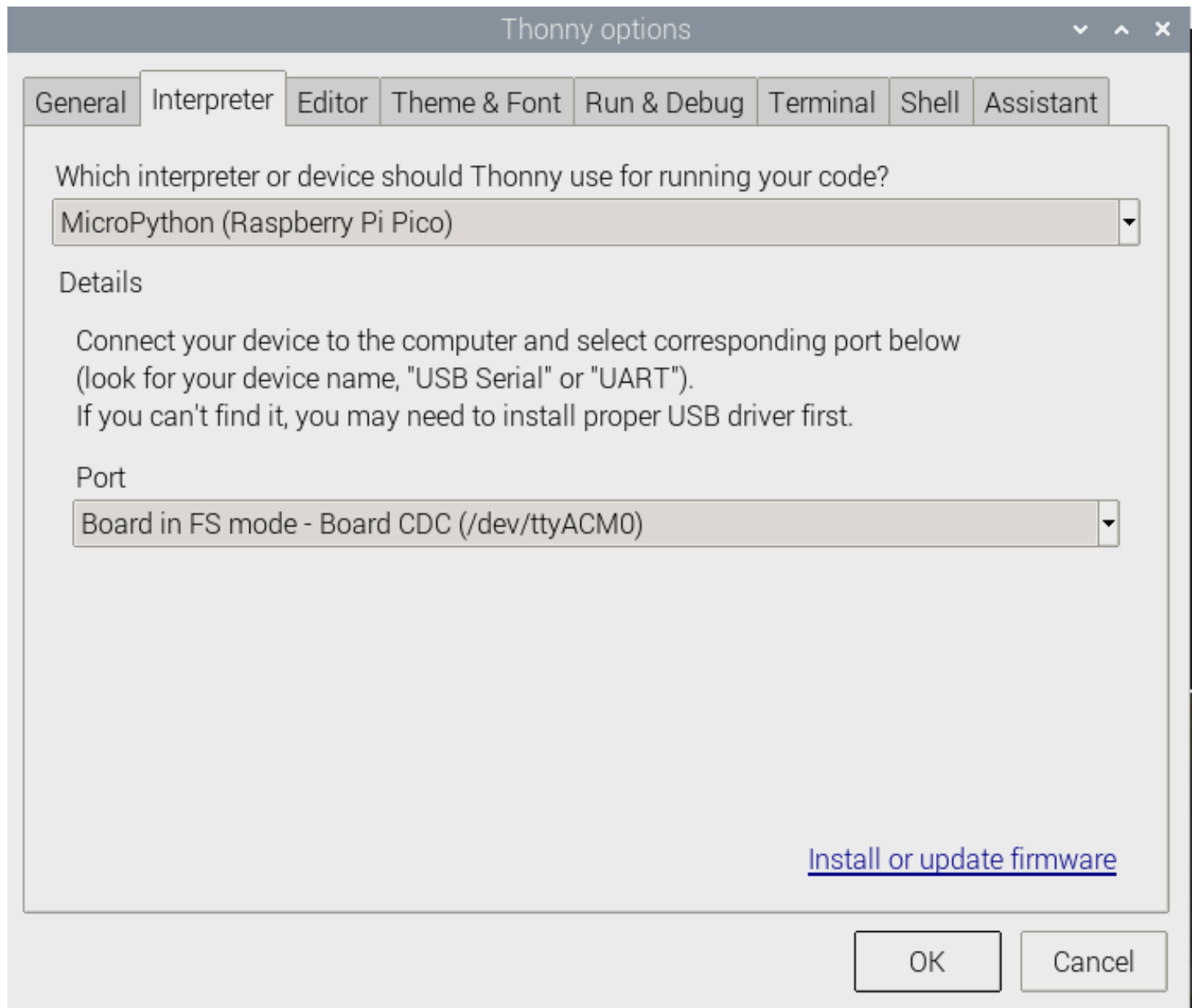
1. Flash the Micropython firmware and copy the pico_micropython_xxxxx.uf2 file into pico (detailed in the Windows tutorial below).

- official firmware download (<https://www.raspberrypi.org/documentation/rp2040/getting-started/#getting-started-with-micropython>)

2. Open Thonny IDE on the Raspberry Pi (click the Raspberry Pi logo -> Programming -> Thonny Python IDE), you can check the version information in Help->About Thonny.

- To make sure your version has Pico support package, also you can click Tools -> Options... -> Interpreter to select MicroPython (Raspberry Pi Pico and ttyACM0 port).

As shown:



(/wiki/File:Pico-lcd-0.96-img-config2.png)

If your current version of Thonny does not have the pico support package, enter the following command to update the Thonny IDE.

```
sudo apt upgrade thonny
```

3. Click File->Open...->python/Pico_Servo_Driver_Code/python/servo.py to run the script.
- As soon as the experimental phenomenon channel is connected to the servo, it will rotate from 0 degrees to 180 degrees, and cycle three times.

Windows



C

- Install in Windows (<https://pico.wiki/index.php/topics/learn/raspberry-pi-pico-c-c-sdk>)

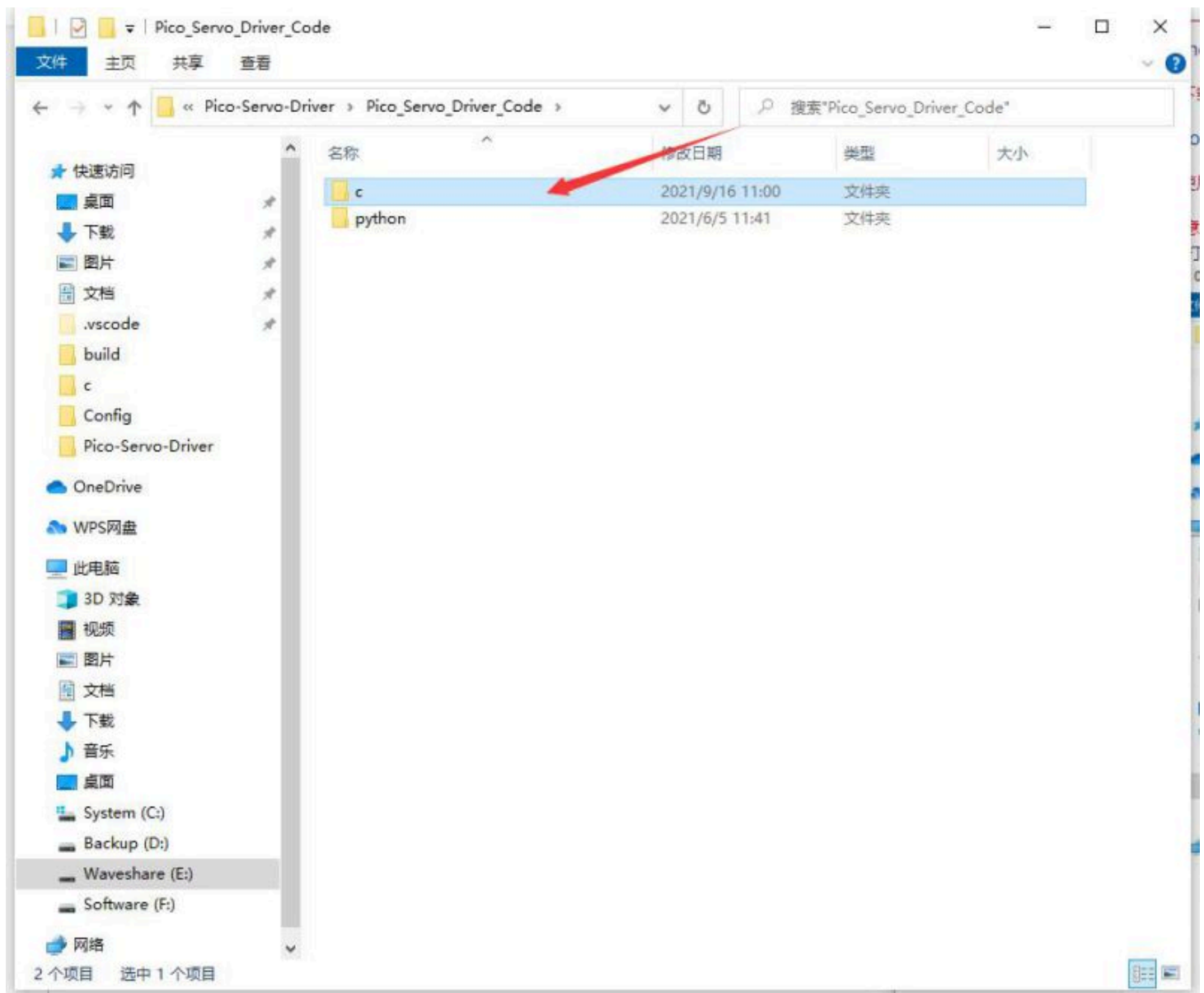
Download the demo

- Pico Servo Driver Demo (https://files.waveshare.com/upload/3/31/Pico_Servo_Driver_Code.7z)

How to Use the Demo

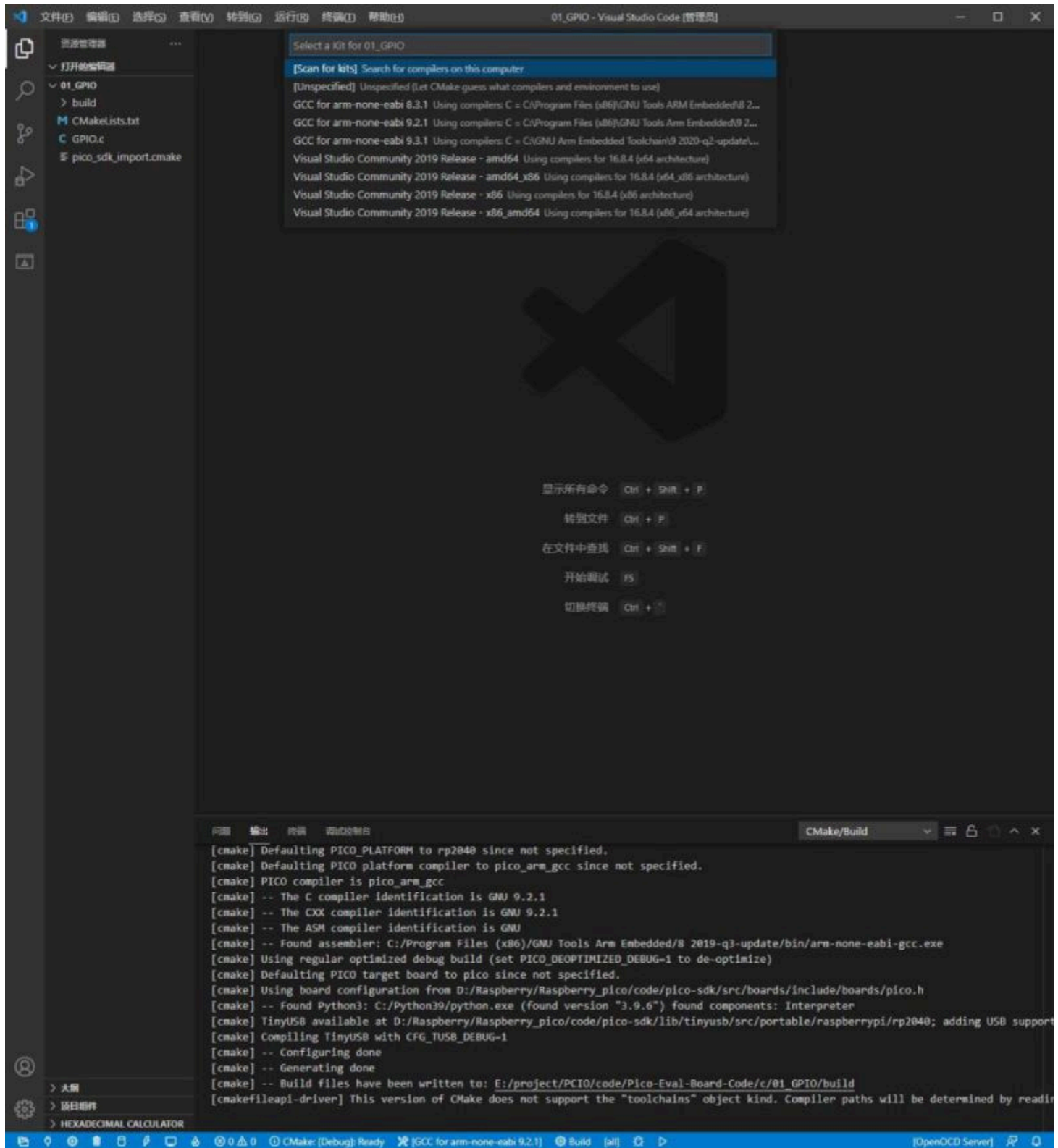
Note: The pictures below are for reference only, the steps are the same.

1. Open the corresponding C program folder.



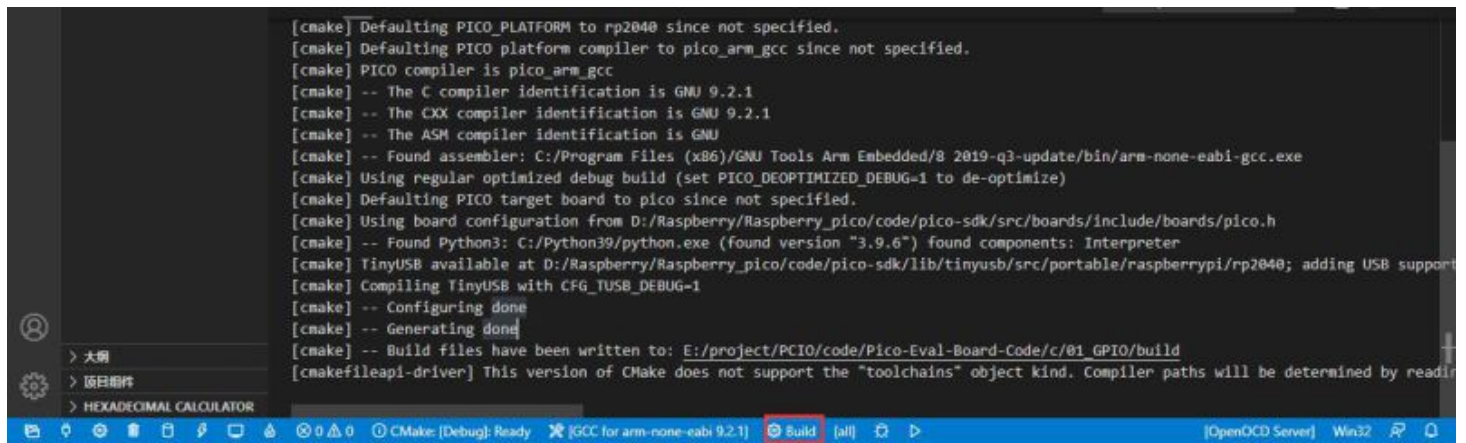
(/wiki/File:Pico_Servo_Driver_1.jpg)

2. Open through the "Vs" coed and select the corresponding compilation tool.



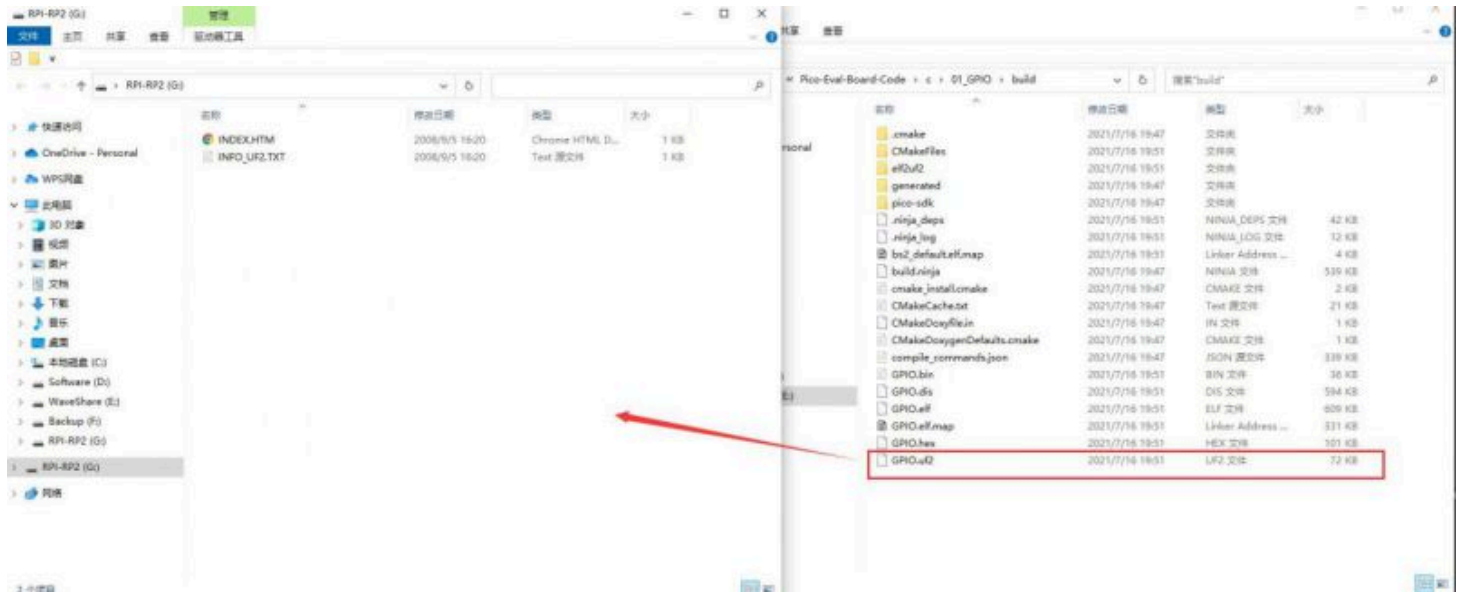
(/wiki/File:Pico-Eval-Board-win-3.jpg)

3. Click the "build" button to compile.



(/wiki/File:Pico-Eval-Board-win-4.jpg)

4. Press the "Reset" button on the Pico-Eval-Board to reset the Pico, first press the BOOTSEL button and then the RUN button and then release the Reset button, the Pico can enter the disk mode without plugging and unplugging the Pico, and the UF2 file under the build file Drag and drop to the RPI-RP2 drive letter.



(/wiki/File:Pico-Eval-Board-win-5.jpg)

5. By now, Pico has started to run the corresponding program.

About the Demo

Underlying hardware interface

- We have the underlying hardware package. Due to different hardware platforms, the internal implementation is different. If you need to know the internal implementation, you can refer to the corresponding directory.

You can see many definitions in DEV_Config.c(.h), in the directory: ...\\c\\lib\\Config.

- data type:

```
#define UBYTE uint8_t
#define UWORD uint16_t
#define UDOUBLE uint32_t
```

- Processing of module initialization and exit:

```
void DEV_Module_Init(void);  
void DEV_Module_Exit(void);
```

- PWM initialization processing:

```
void PWM_initialization();
```

- Interrupt handler:

```
void on_pwm_wrap();
```

- Define the channel used:

```
#define CHANNE_N 0xFFFF // 0x0001 means 0 channel is open, 0x0000 means all channels are closed
```

- Rotation angle:

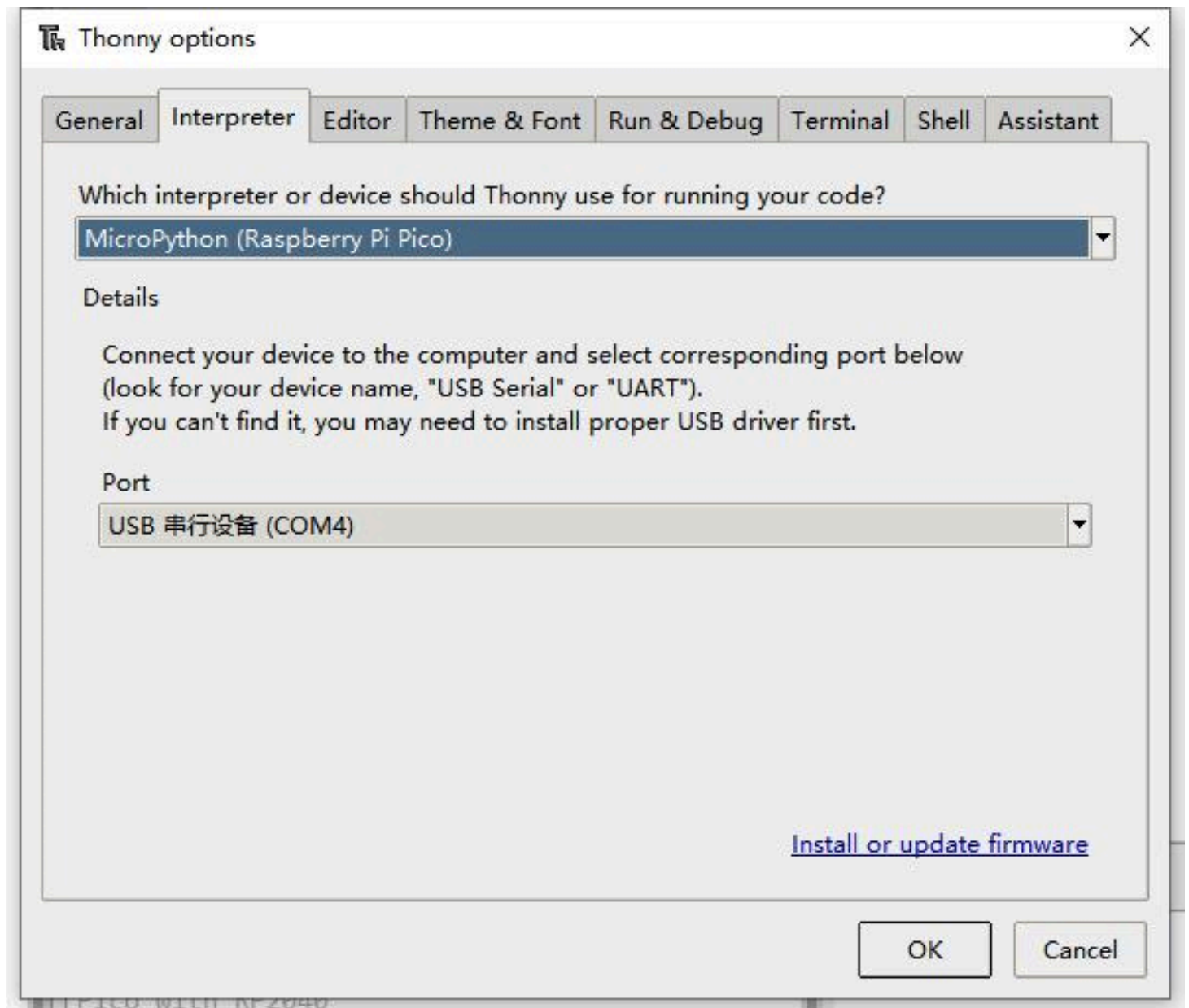
```
#define ROTATE_0 1700 //Rotate to 0° position  
#define ROTATE_45 3300 //Rotate to 45° position  
#define ROTATE_90 4940 //Rotate to 90° position  
#define ROTATE_135 6600 //Rotate to 135° position  
#define ROTATE_180 8250 //Rotate to 180° position
```

Python

1. Press and hold the BOOTSET button on the Pico board, connect the Pico to the USB port of the computer through the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).
2. Download the pico_micropython_xxxxx.uf2 file and copy it to the recognized removable disk (RPI-RP2)

official firmware download (<https://www.raspberrypi.org/documentation/rp2040/getting-started/#getting-started-with-micropython>).

3. Open Thonny IDE (Note: Use the latest version of Thonny, otherwise there is no Pico support package, the latest version under Windows is v3.3.3).
4. Click Tools->Settings->Interpreter, and select Pico and the corresponding port as shown in the figure.



(/wiki/File:Pico-lcd-0.96-img-config.png)

5. File->Open->servo.py, click to run, as shown in the figure, the program has been run:

```
MicroPython v1.13-290-g556ae7914 on 2021-01-21; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
```

(/wiki/File:Pico-lcd-0.96-img-run.png)

The experimental phenomenon is the same as the c program, and will not be repeated here.

Resource

Document

- Schematic (https://files.waveshare.com/upload/0/06/Pico_Servo_Driver_Sch.pdf)

Demo codes

- Demo codes (https://files.waveshare.com/upload/3/30/Pico_Servo_Driver_Code.zip)

Development Softwares

- Zimo221.7z (<https://files.waveshare.com/upload/c/c6/Zimo221.7z>)
- Image2Lcd.7z (<https://files.waveshare.com/upload/3/36/Image2Lcd.7z>)
-
-
- Thonny Python IDE (Windows V3.3.3) (<https://files.waveshare.com/upload/7/73/Thonny-3.3.3.zip>)

Pico Getting Started

Firmware Download

- MicroPython Firmware Download [\[Expand\]](#)
- C_Blink Firmware Download [\[Expand\]](#)

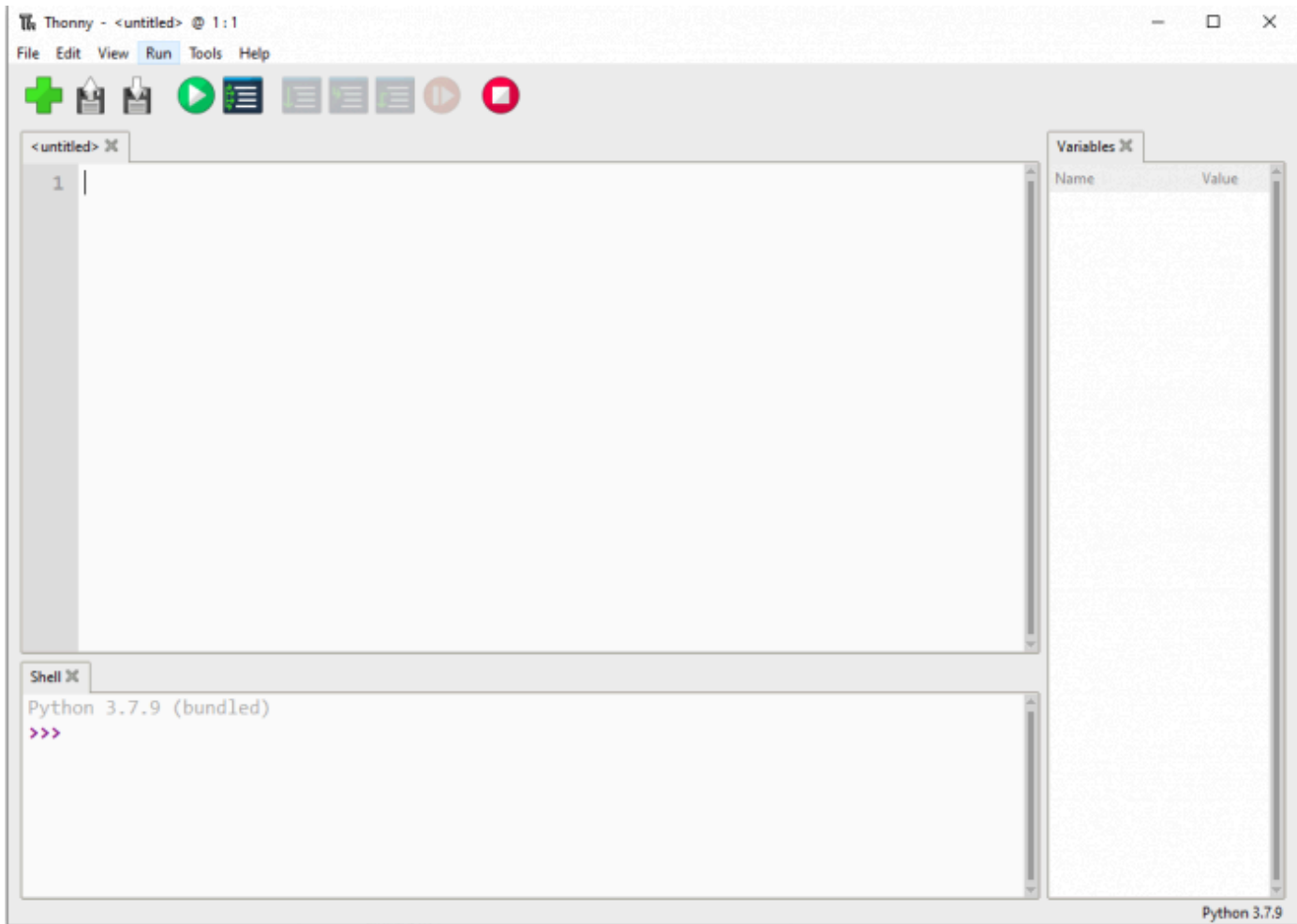
Introduction

MicroPython Series

Install Thonny IDE

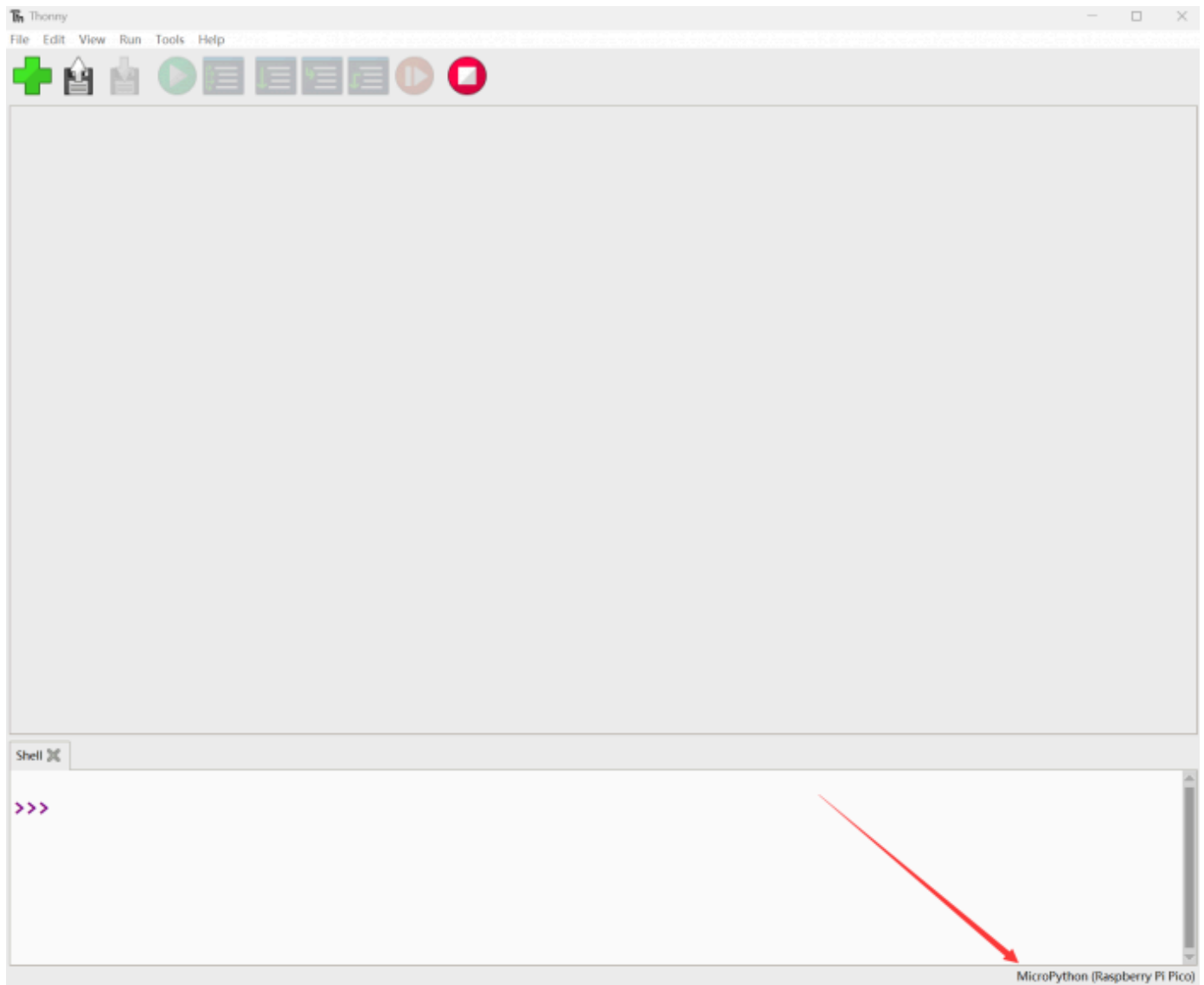
In order to facilitate the development of Pico/Pico2 boards using MicroPython on a computer, it is recommended to download the Thonny IDE

- Download Thonny IDE and follow the steps to install, the installation packages are all Windows versions, please refer to Thonny's official website for other versions
 - Thonny IDE official download link (<https://github.com/thonny/thonny/releases/download/v3.3.3/thonny-3.3.3.exe>)
 - Thonny IDE download link (<https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip>)
 - Thonny official website (<https://thonny.org/>)
- After installation, the language and motherboard environment need to be configured for the first use. Since we are using Pico/Pico2, pay attention to selecting the Raspberry Pi option for the motherboard environment

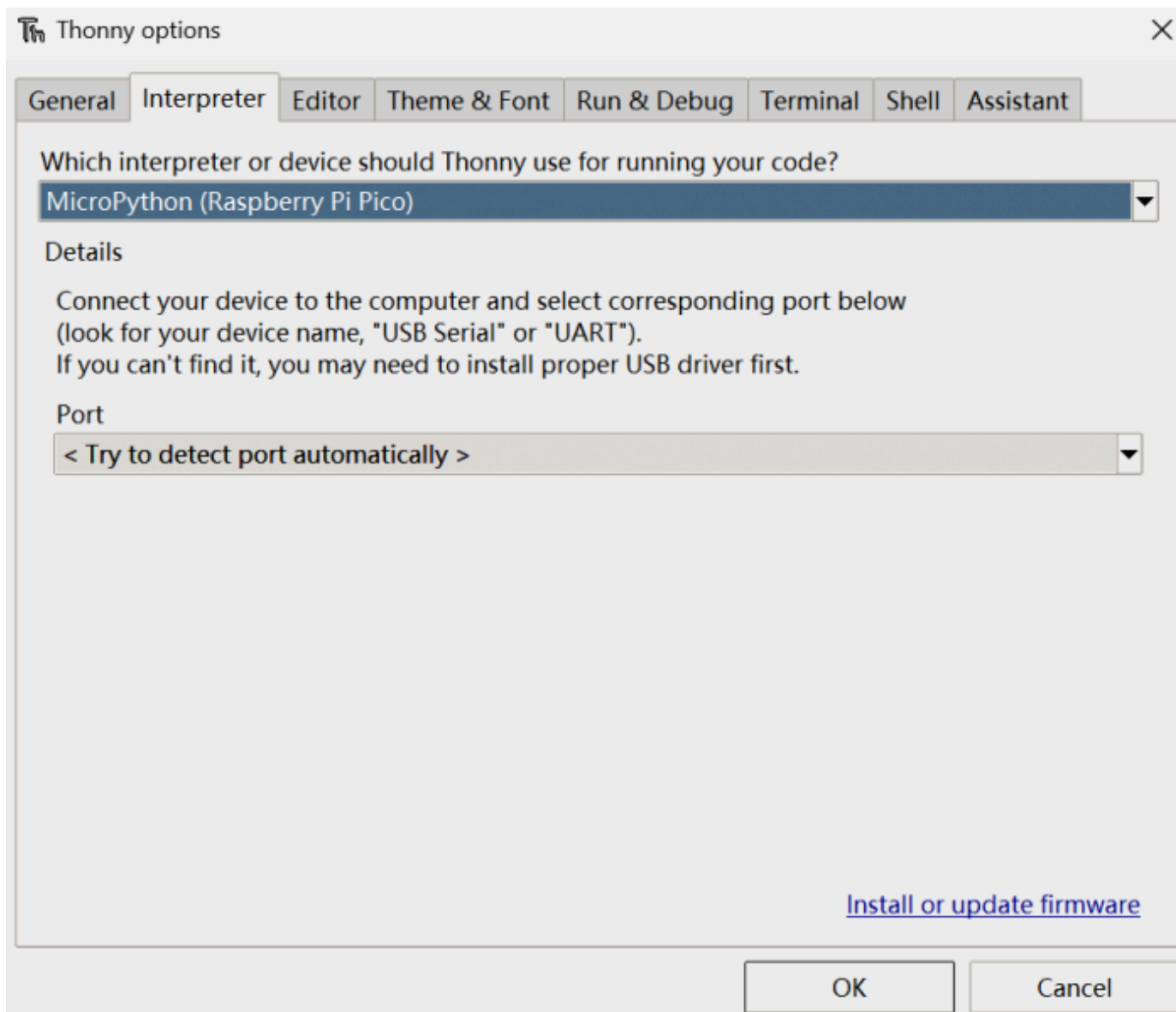


(/wiki/File:Pico-R3-Tonny1.png)

- Configure MicroPython environment and choose Pico/Pico2 port
 - Connect Pico/Pico2 to your computer first, and in the lower right corner of Thonny left-click on the configuration environment option --> select Configure interpreter
 - In the pop-up window, select MicroPython (Raspberry Pi Pico), and choose the corresponding port



(/wiki/File:700px-Raspberry-Pi-Pico-Basic-Kit-M-2.png)



(/wiki/File:700px-Raspberry-Pi-Pico-Basic-Kit-M-3.png)

Flash Firmware

- Click OK to return to the Thonny main interface, download the corresponding firmware library and flash it to the device, and then click the Stop button to display the current environment in the Shell window
- **Note: Flashing the Pico2 firmware provided by Micropython may cause the device to be unrecognized, please use the firmware below or in the package**
 - Pico firmware library (<https://files.waveshare.com/wiki/common/Rp2-pico-20210418-v1.15.7z>)
 - Pico2 firmware library (https://files.waveshare.com/wiki/common/RPI_PICO2-20240809-v1.24.0.zip)
- How to download the firmware library for Pico/Pico2 in windows: After holding down the BOOT button and connecting to the computer, release the BOOT button, a removable disk will appear on the computer, copy the firmware library into it
- How to download the firmware library for RP2040/RP2350 in windows: After connecting to the computer, press the BOOT key and the RESET key at the same time, release the RESET key first and then release the BOOT key, a removable disk will appear on the computer, copy the firmware library into it (you can also use the Pico/Pico2 method)



(/wiki/File:Raspberry-Pi-Pico2-Python.png)

MicroPython Series

C/C++ Series

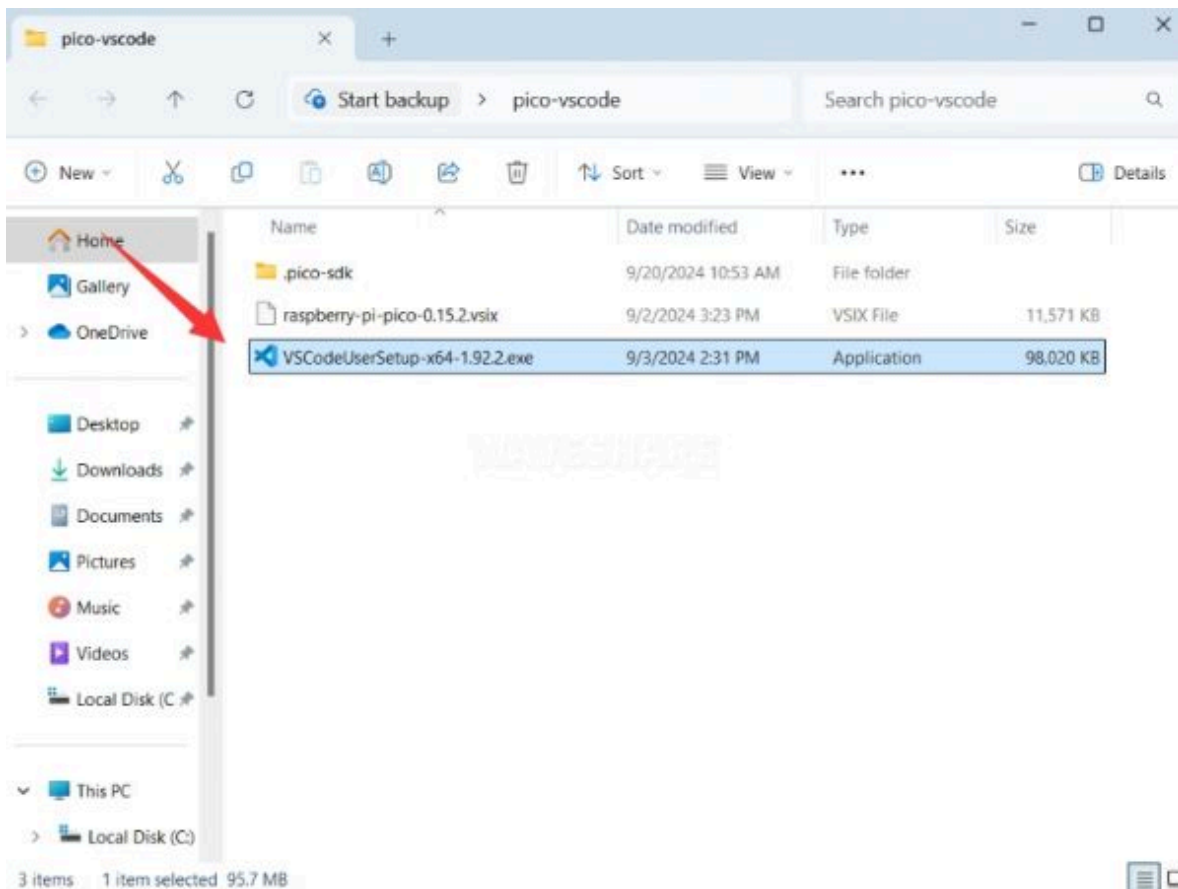
For C/C++, it is recommended to use Pico VS Code for development. This is a Microsoft Visual Studio Code extension designed to make it easier for you to create, develop, and debug projects for the Raspberry Pi Pico series development boards. No matter if you are a beginner

or an experienced professional, this tool can assist you in developing Pico with confidence and ease. Here's how to install and use the extension.

- Official website tutorial: <https://www.raspberrypi.com/news/pico-vscode-extension/> (<https://www.raspberrypi.com/news/pico-vscode-extension/>)
- This tutorial is suitable for Raspberry Pi Pico, Pico2 and the RP2040 and RP2350 series development boards developed by WaveShare
- The development environment defaults to Windows11. For other environments, please refer to the official tutorial for installation

Install VSCode

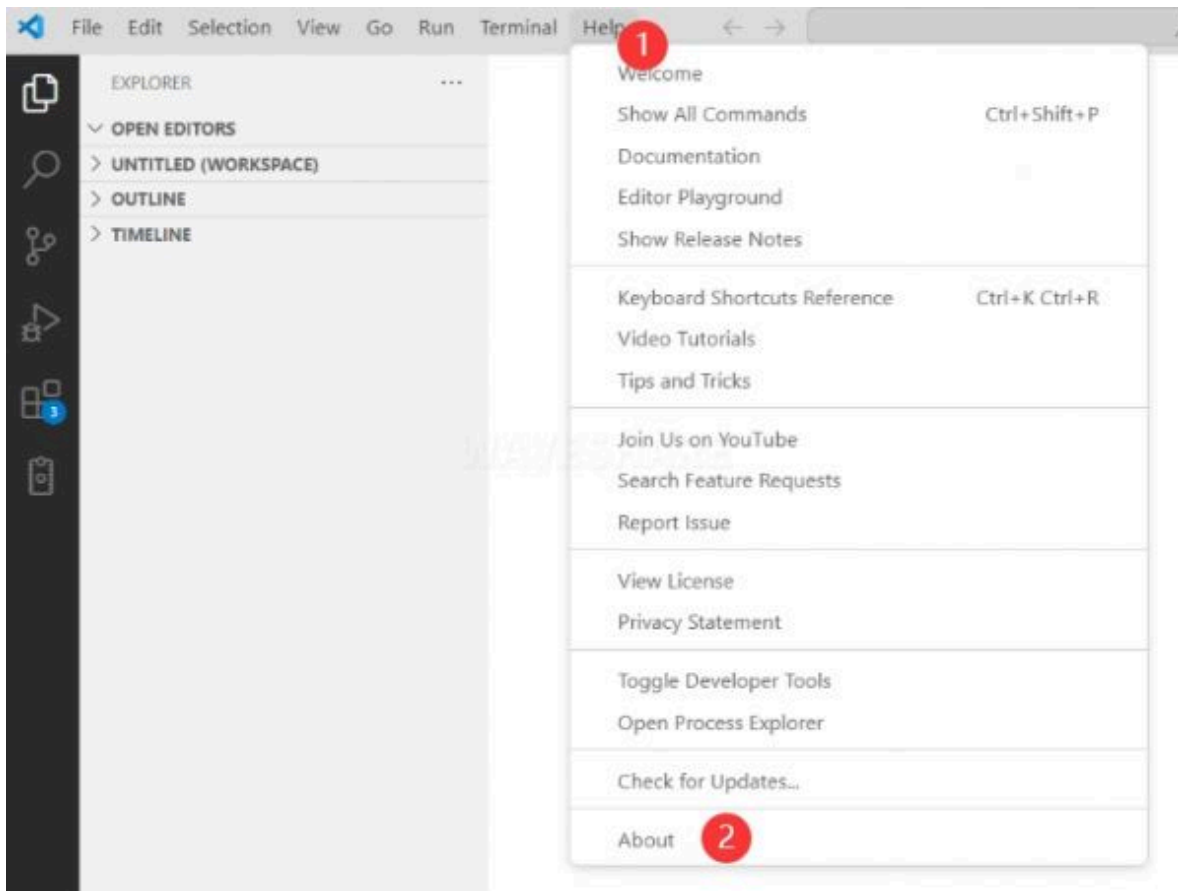
1. First, click to download pico-vscode package (<https://drive.google.com/file/d/18-KDNrQII0KuTMdS6W5ibIUgaGm3FbVJ/view?usp=sharing>), unzip and open the package, double-click to install VSCode



(/wiki/File:Pico-

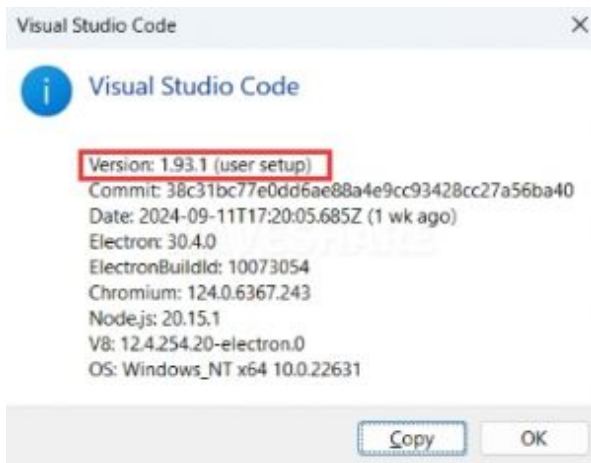
vscode-1.JPG)

Note: If vscode is installed, check if the version is v1.87.0 or later



(/wiki/File:Pico-

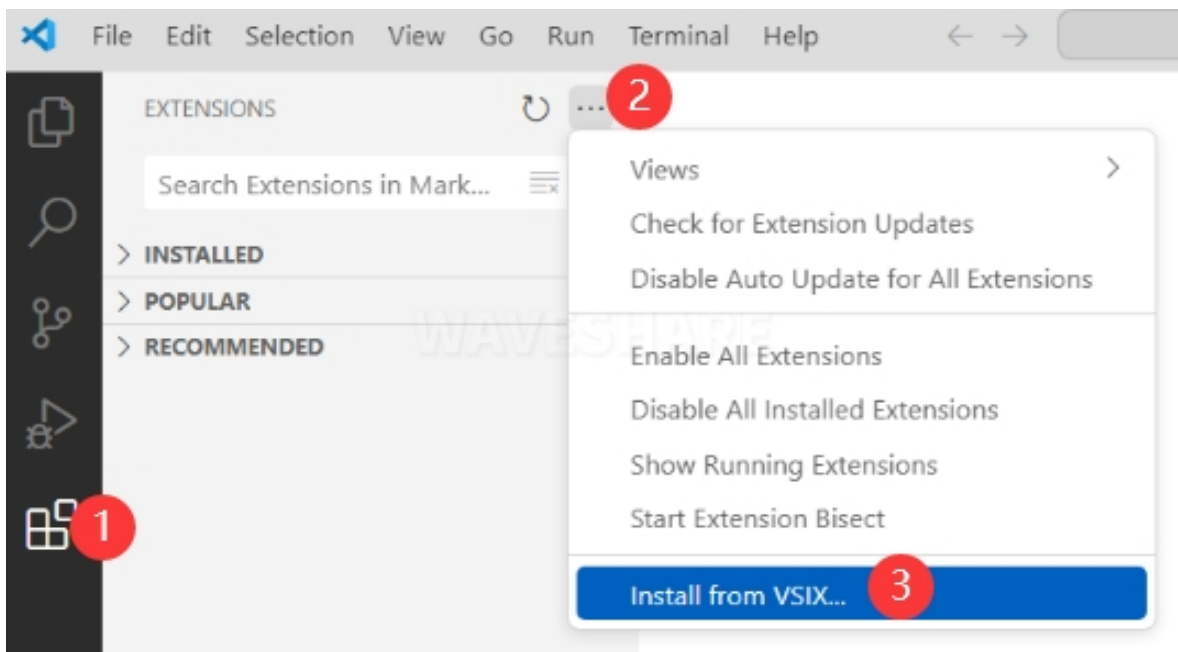
vscode-2.JPG)



(/wiki/File:Pico-vscode-3.JPG)

Install Extension

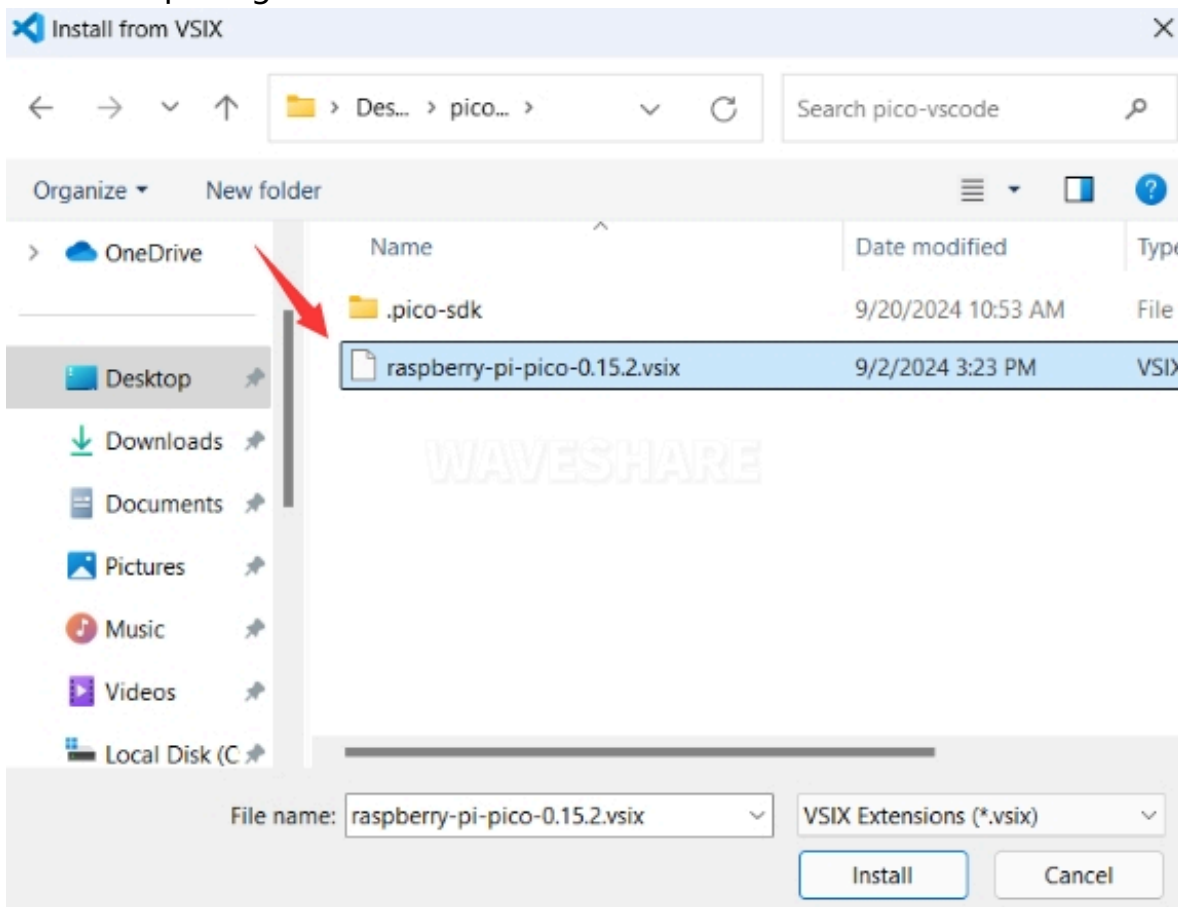
1. Click Extensions and select Install from VSIX



(/wiki/File:Pico-

vscode-4.JPG)

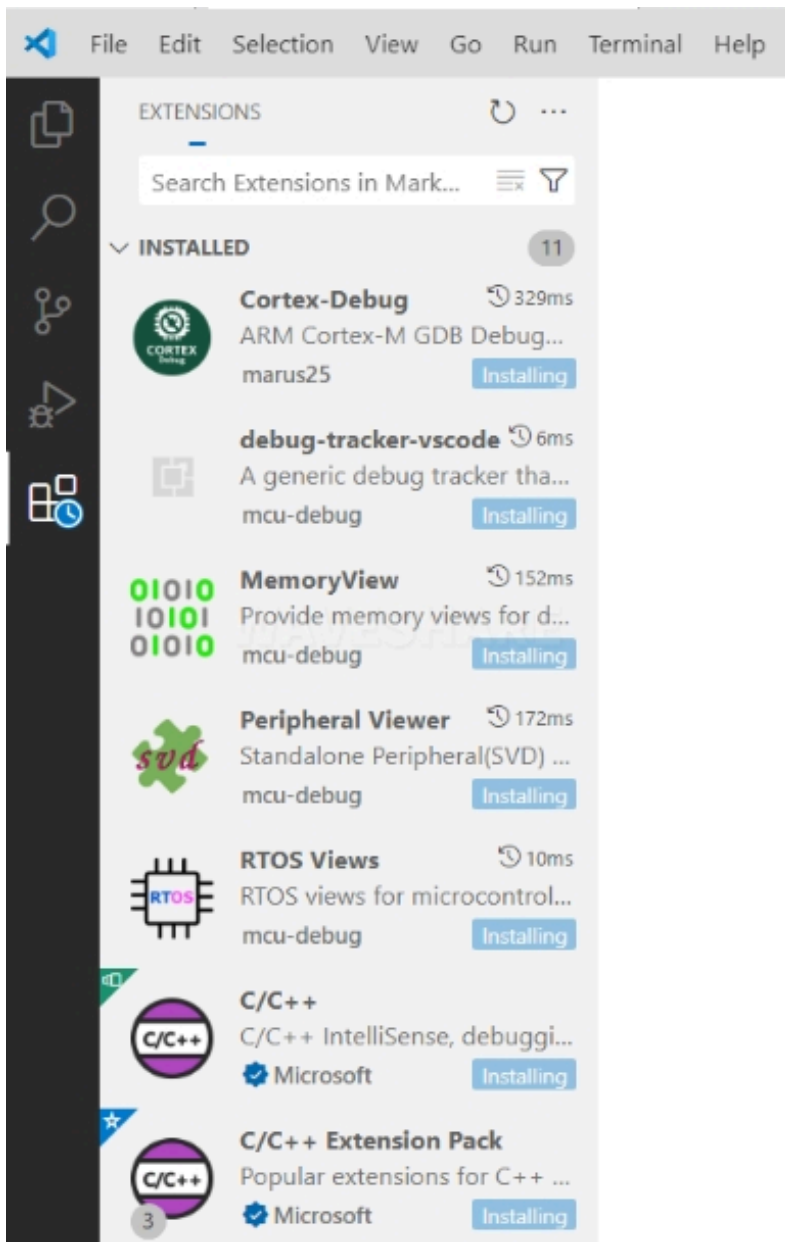
2. Select the package with the vsix suffix and click Install



(/wiki/File:Pico-

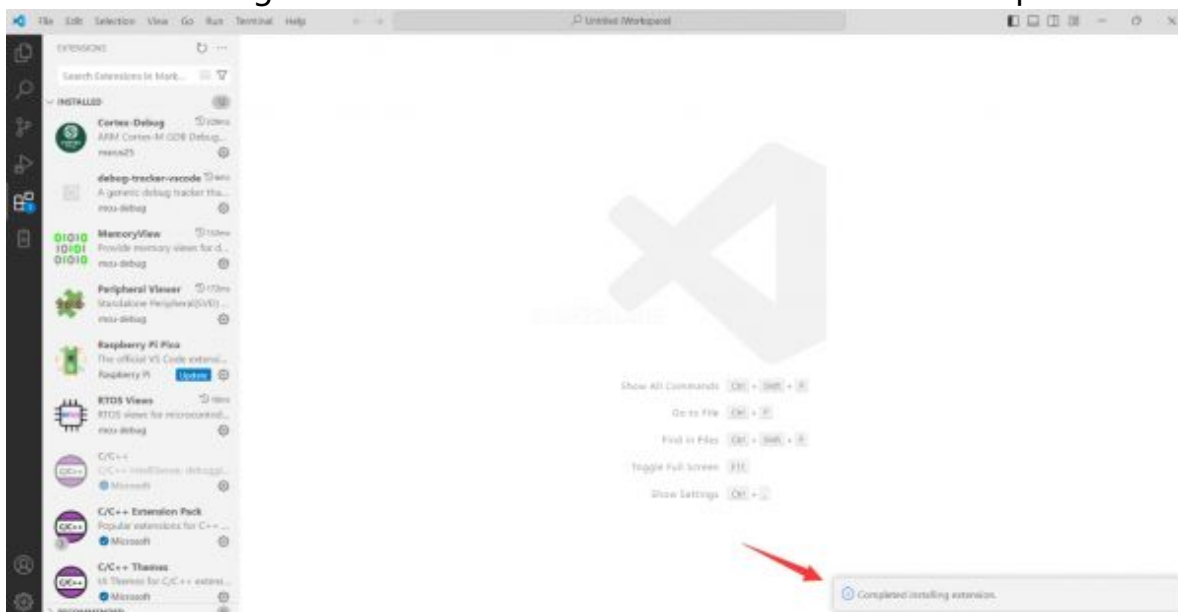
vscode-5.JPG)

3. Then vscode will automatically install raspberry-pi-pico and its dependency extensions, you can click Refresh to check the installation progress



(/wiki/File:Pico-vscode-6.JPG)

4. The text in the right lower corner shows that the installation is complete. Close VSCode

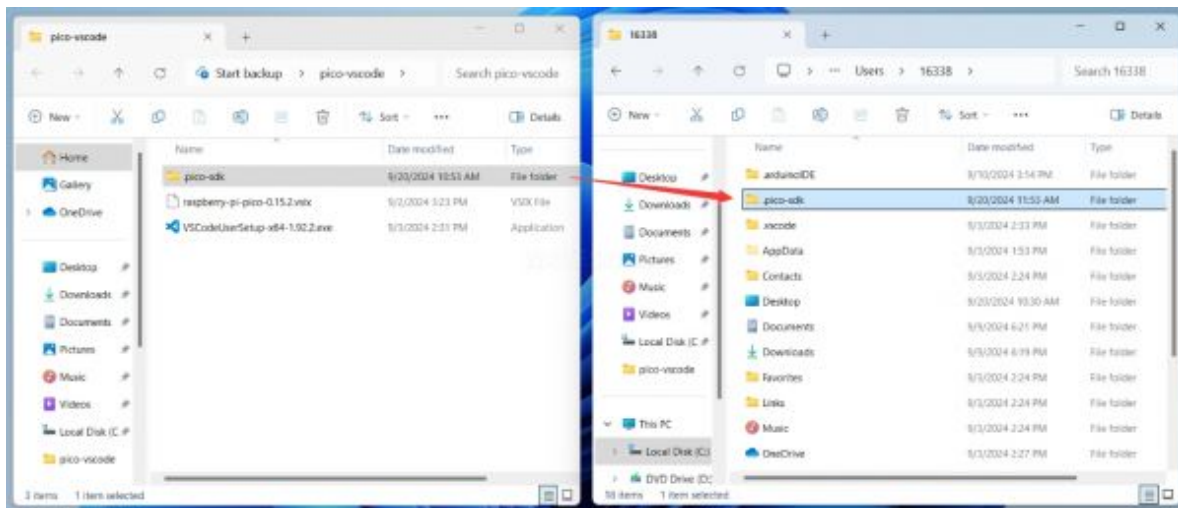


(/wiki/File:Pico-

vscode-7.JPG)

Configure Extension

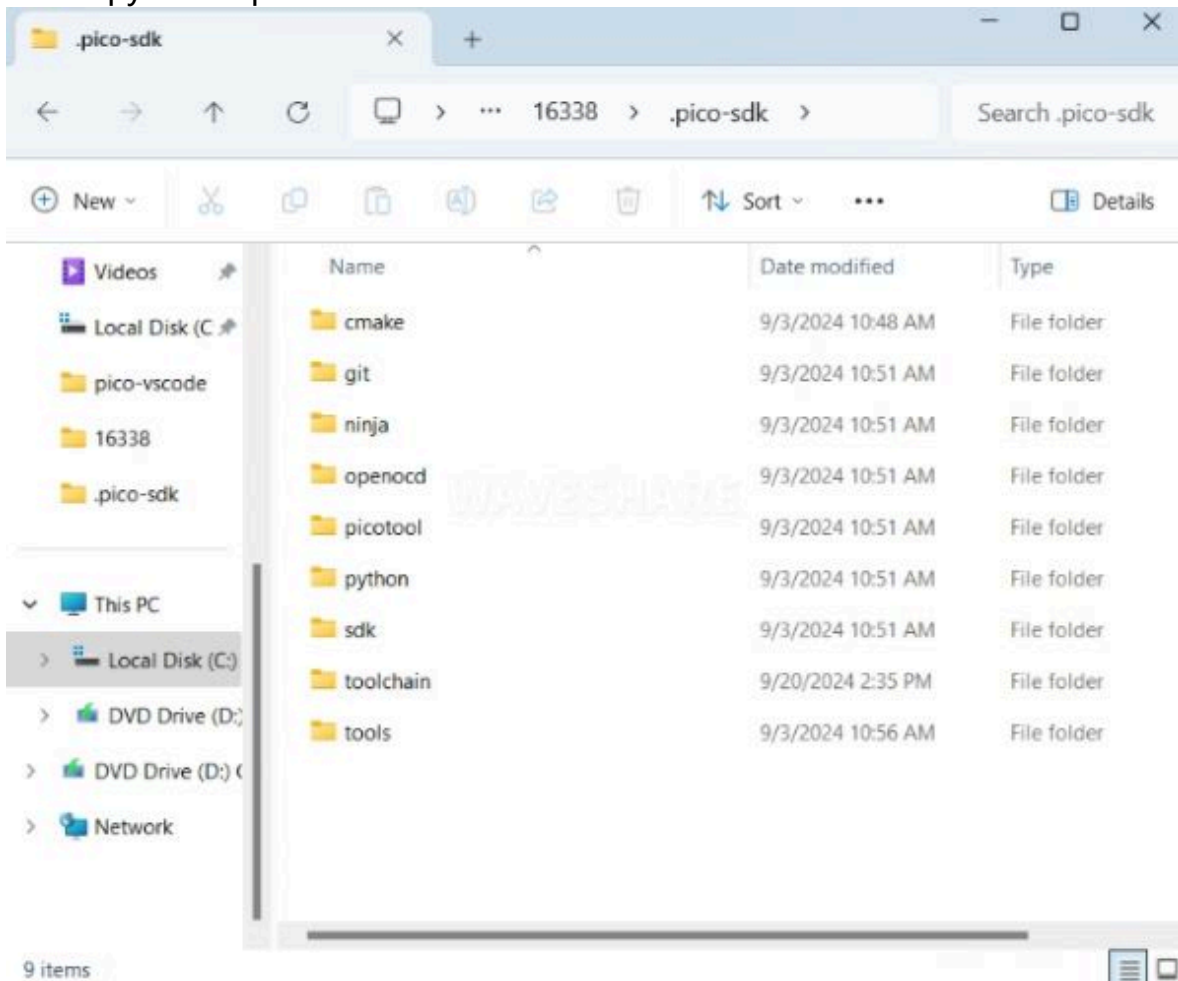
1. Open directory C:\Users\username and copy the entire .pico-sdk to that directory



(/wiki/File:Pico-

vscode-8.JPG)

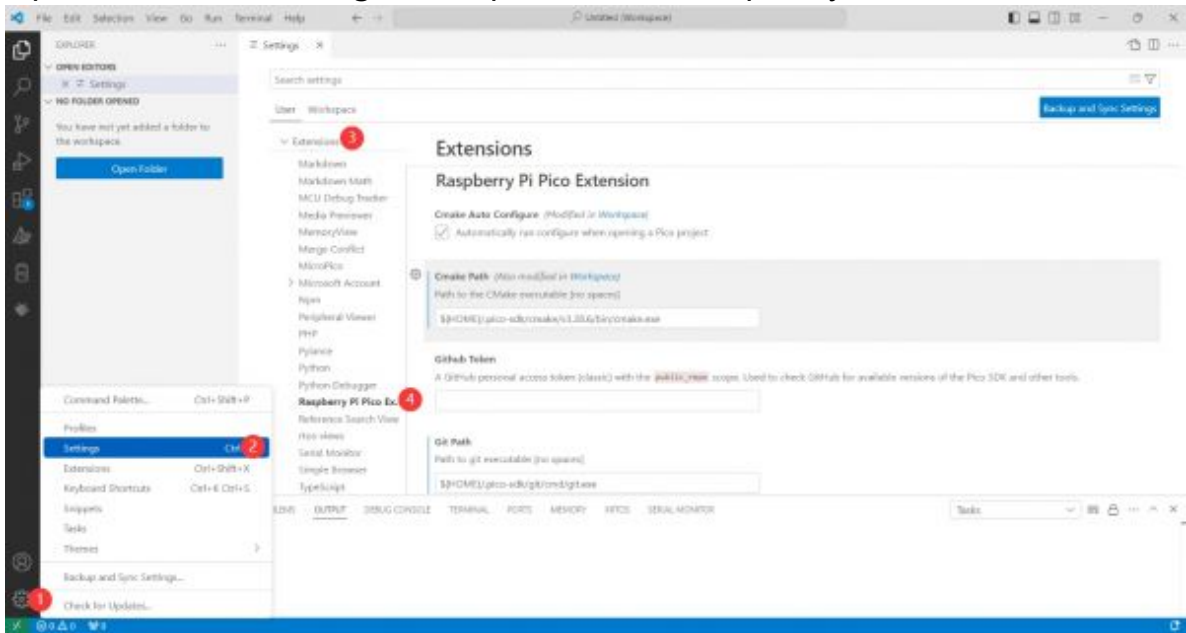
2. The copy is completed



(/wiki/File:Pico-

vscode-9.JPG)

3. Open vscode and configure the paths for the Raspberry Pi Pico extensions



(/wiki/File:Pico-

vscode-10.JPG)

The configuration is as follows:

Cmake Path:

`${HOME}/.pico-sdk/cmake/v3.28.6/bin/cmake.exe`

Git Path:

`${HOME}/.pico-sdk/git/cmd/git.exe`

Ninja Path:

`${HOME}/.pico-sdk/ninja/v1.12.1/ninja.exe`

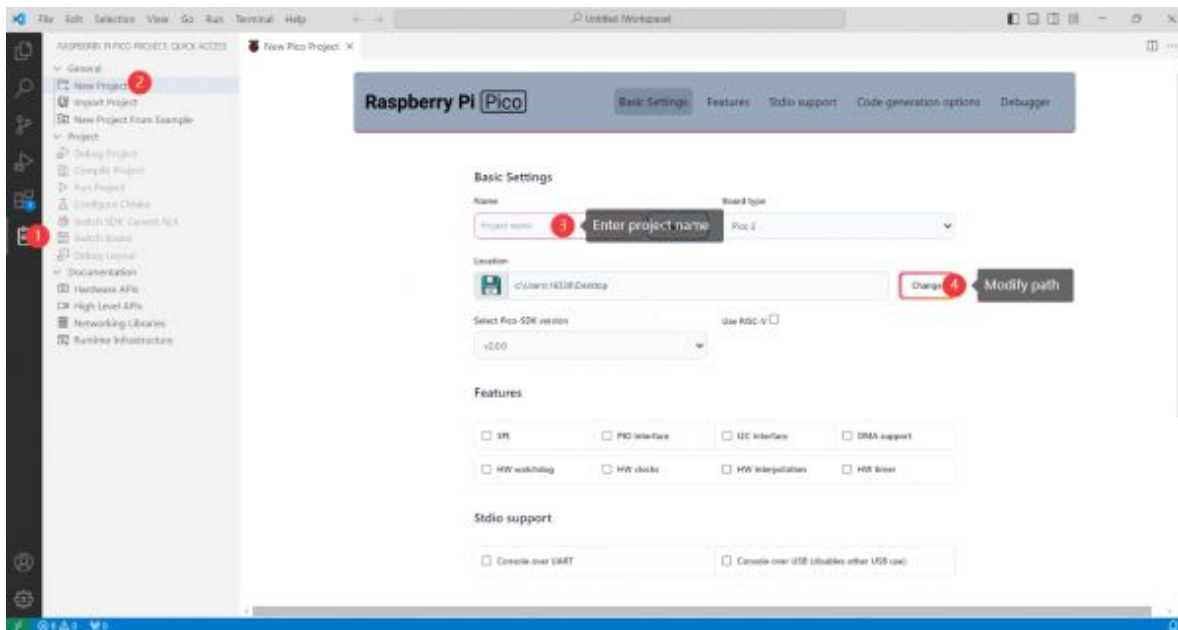
Python3 Path:

`${HOME}/.pico-sdk/python/3.12.1/python.exe`

New Project

1. The configuration is complete, create a new project, enter the project name, select the path, and click Create to create the project

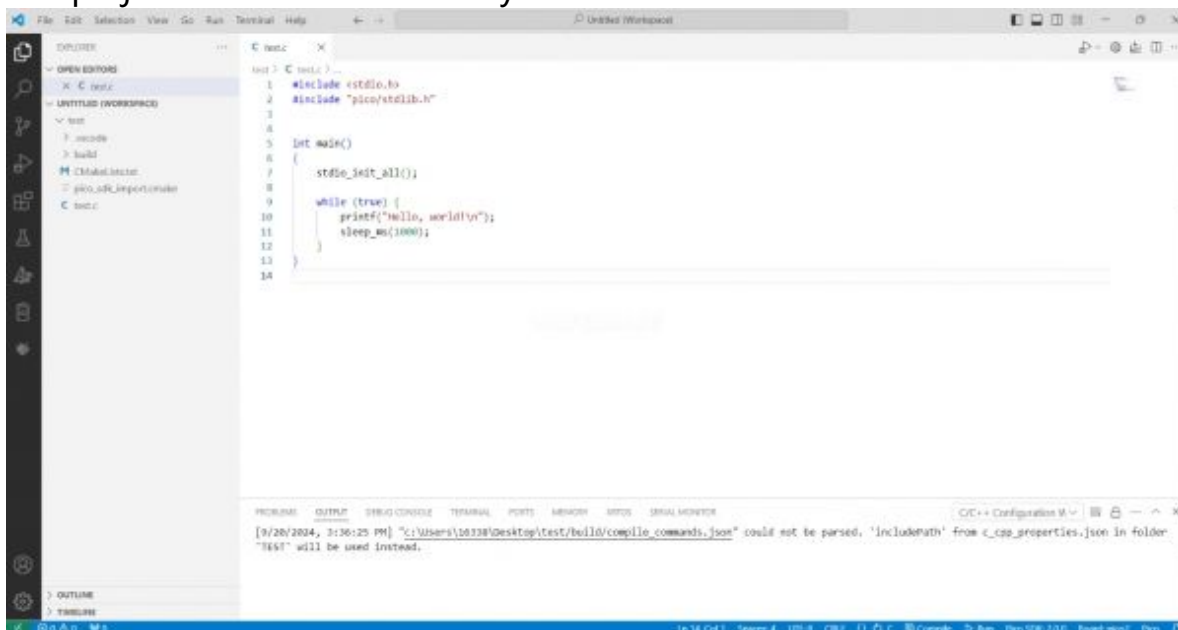
To test the official example, you can click on the Example next to the project name to select



(/wiki/File:Pico-

vscode-11.JPG)

2. The project is created successfully

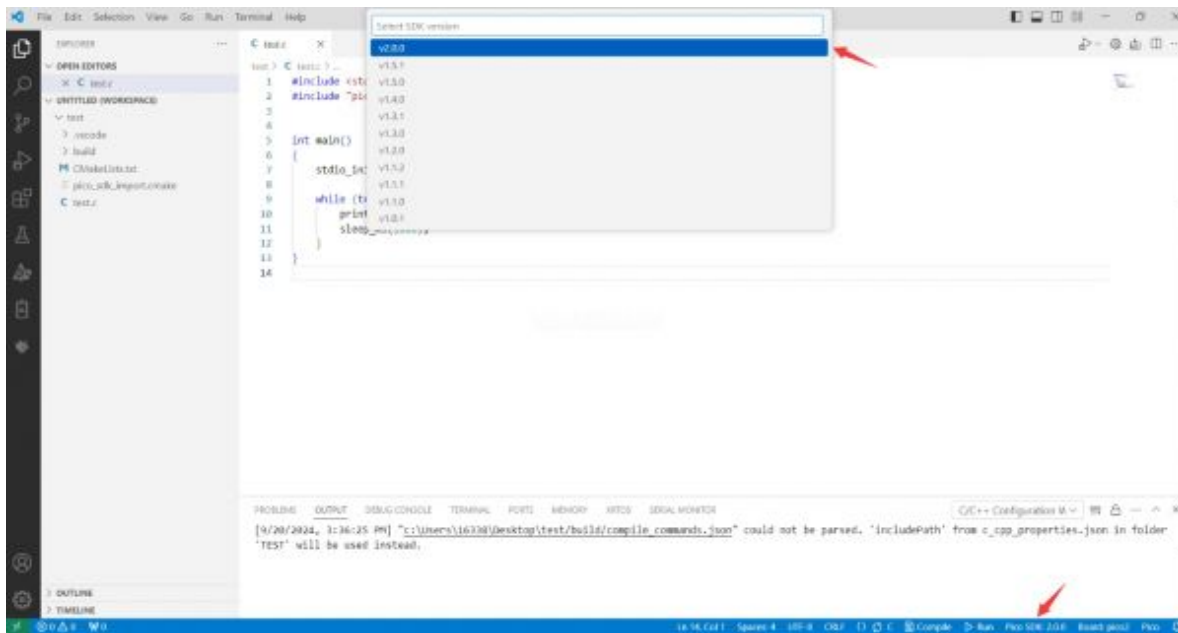


(/wiki/File:Pico-

vscode-12.JPG)

Compile Project

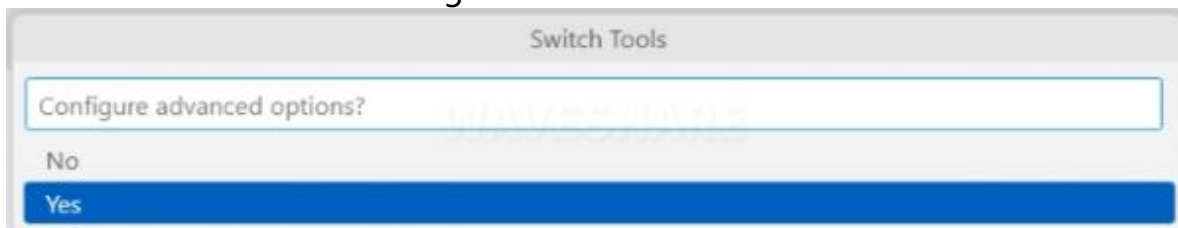
1. Select the SDK version



(/wiki/File:Pico-

vscode-13.JPG)

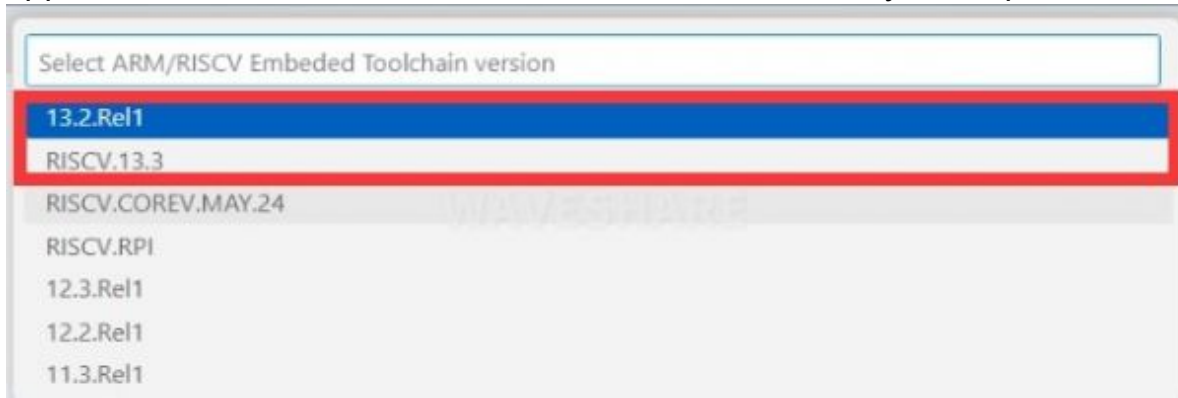
2. Select Yes for advanced configuration



(/wiki/File:Pico-

vscode-14.JPG)

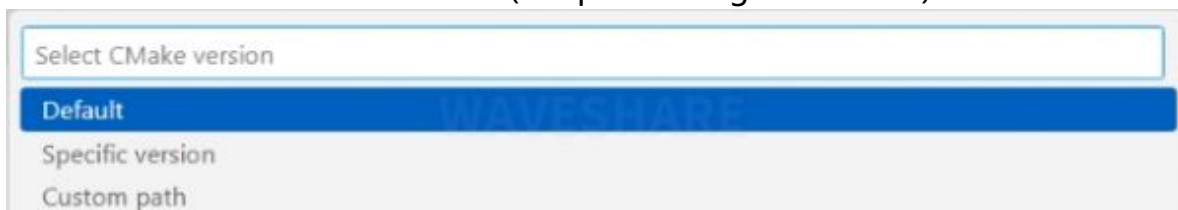
3. Choose the cross-compilation chain, 13.2.Rel1 is applicable for ARM cores, RISC.V13.3 is applicable for RISC.V cores. You can select either based on your requirements



(/wiki/File:Pico-

vscode-15.JPG)

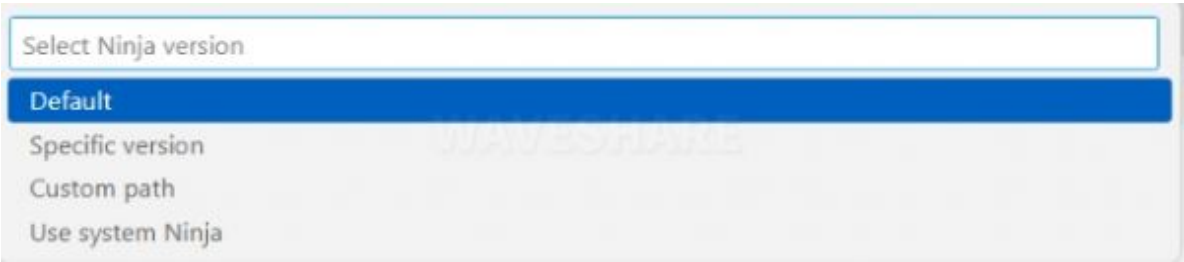
4. Select Default for CMake version (the path configured earlier)



(/wiki/File:Pico-

vscode-16.JPG)

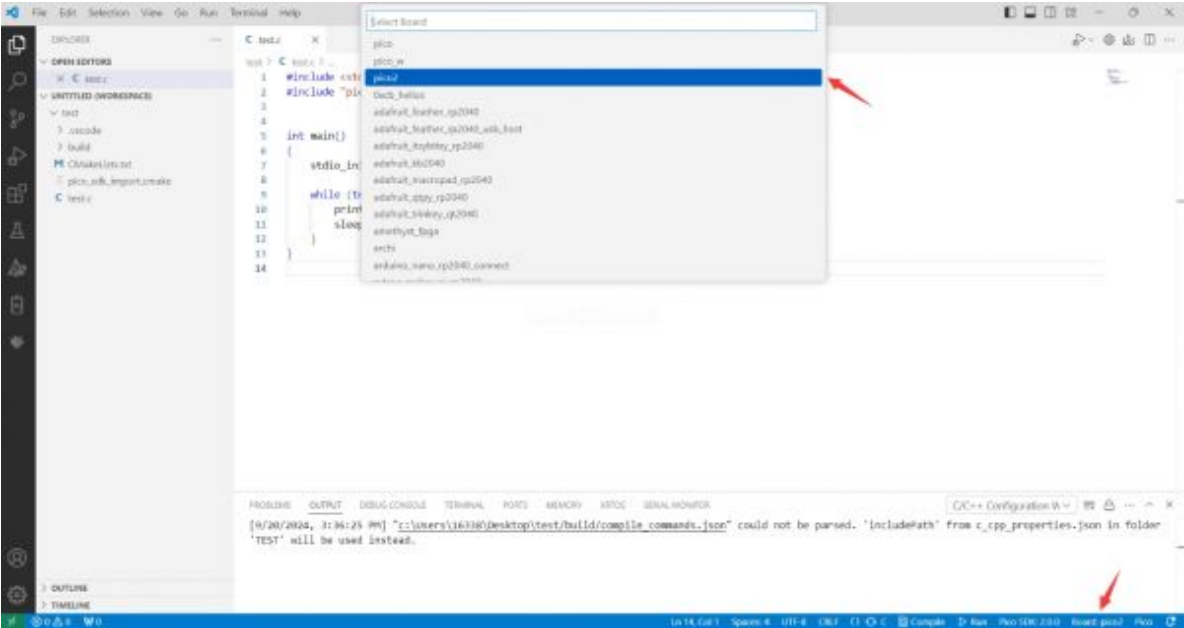
5. Select Default for Ninja version



(/wiki/File:Pico-

vscode-17.JPG)

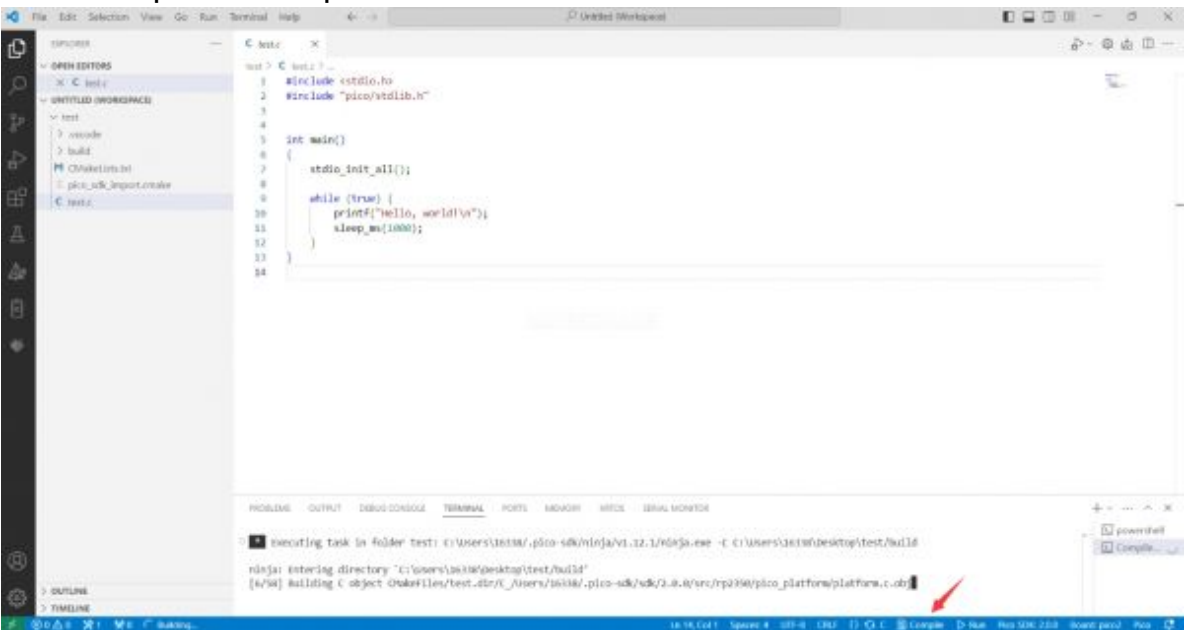
6. Select the development board



(/wiki/File:Pico-

vscode-18.JPG)

7. Click Compile to compile



(/wiki/File:Pico-

vscode-19.JPG)

8. The uf2 format file is successfully compiled



(/wiki/File:Pico-

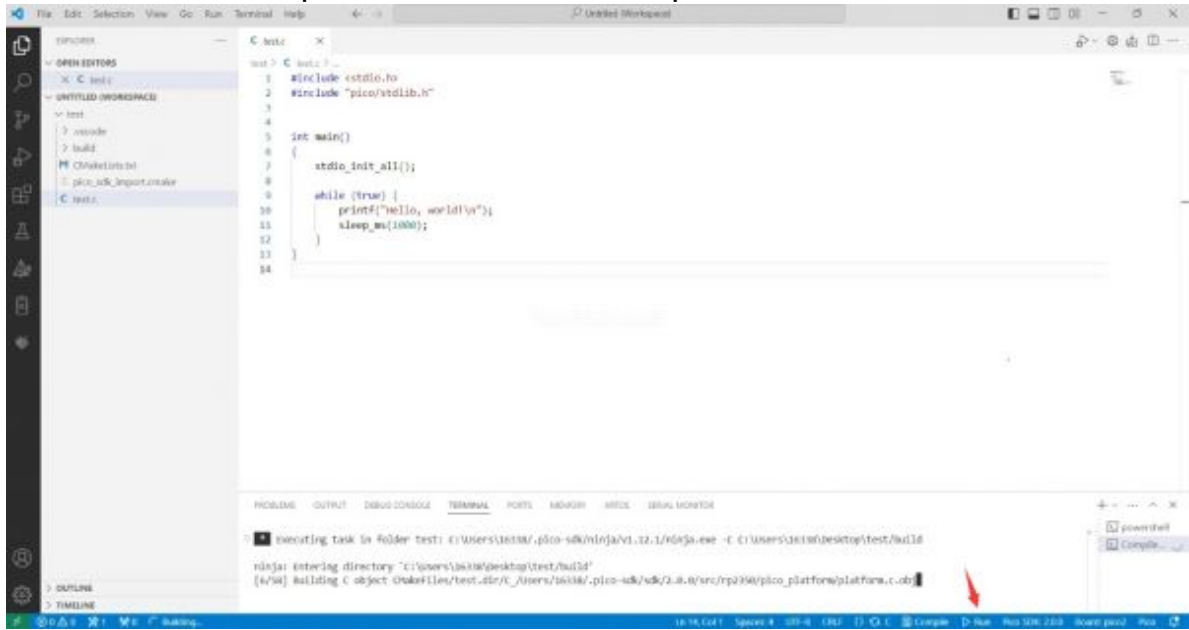
vscode-20.JPG)

Flash Firmware

Here are two methods for flashing firmware

1. Flash firmware using the pico-vscode plugin

Connect the development board to the computer, click Run to flash the firmware directly



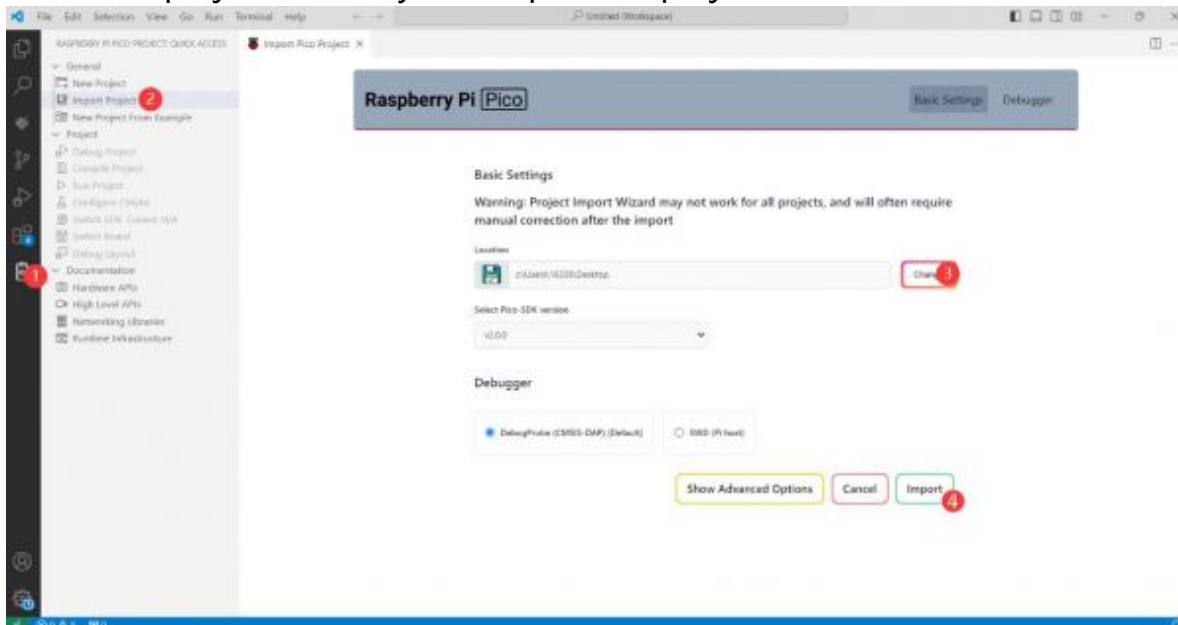
(/wiki/File:600px-Pico-vscode-24.jpg)

2. Flash the firmware manually

1. Press and hold the Boot button
2. Connect the development board to the computer
3. Then the computer will recognize the development board as a USB device.
4. Copy the .uf2 file to the USB drive, and the device will automatically restart, indicating successful program flashing.

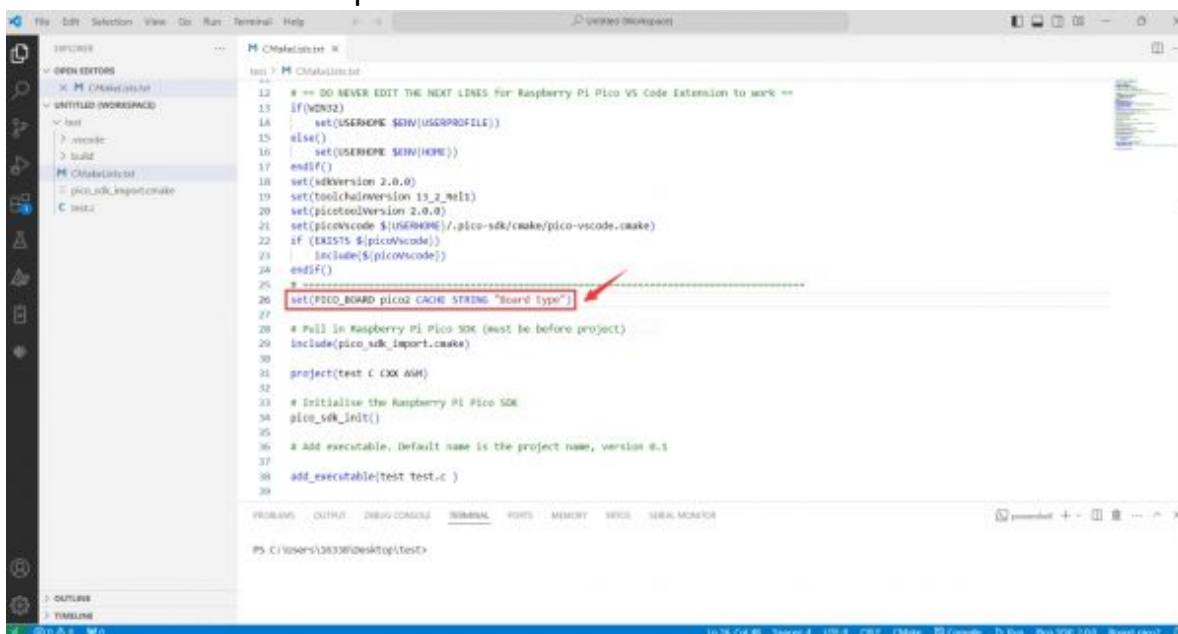
Import Project

1. Select the project directory and import the project



(/wiki/File:600px-Pico-vscode-23.jpg)

2. The Cmake file of the imported project cannot have Chinese (including comments), otherwise the import may fail
3. To import your own project, you need to add a line of code to the Cmake file to switch between pico and pico2 normally, otherwise even if pico2 is selected, the compiled firmware will still be suitable for pico



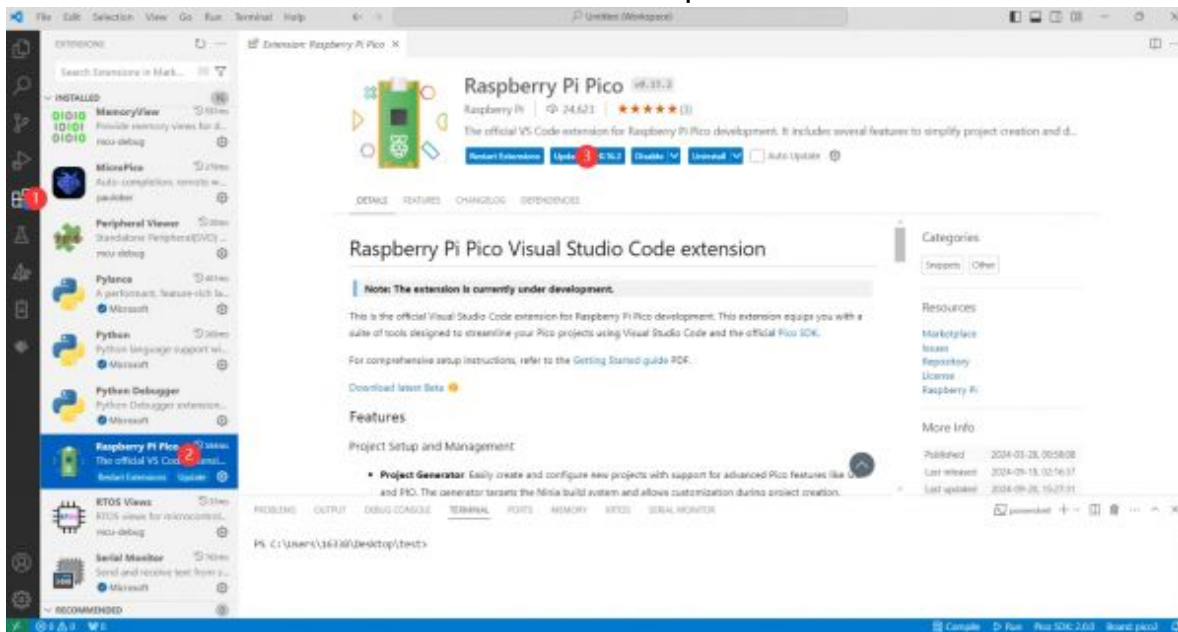
(/wiki/File:Pico-

vscode-21.JPG)

```
set(PICO_BOARD pico CACHE STRING "Board type")
```

Update Extension

1. The extension version in the offline package is 0.15.2, and you can also choose to update to the latest version after the installation is complete



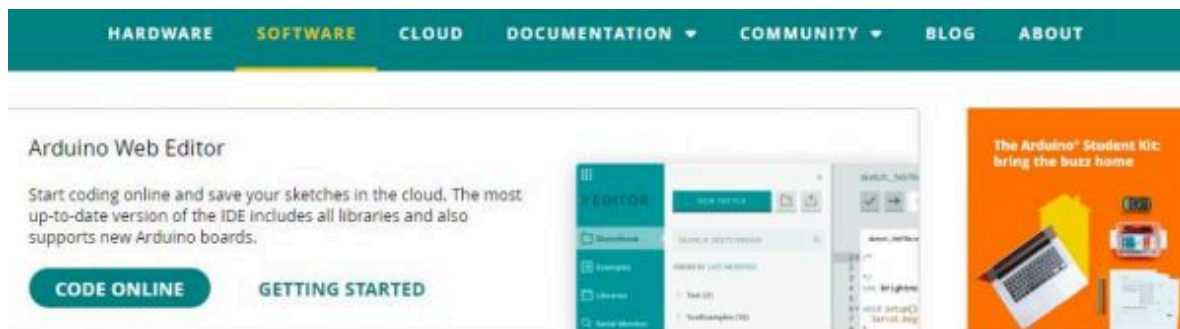
(/wiki/File:Pico-

vscode-22.JPG)

Arduino IDE Series

Install Arduino IDE

1. First, go to Arduino official website (<https://www.arduino.cc/>) to download the installation package of the Arduino IDE.



Downloads



Arduino IDE 2.0.0

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file
Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)
macOS 10.14: "Mojave" or newer, 64 bits

(/wiki/File:600px-Arduino%E4%B8%8B%E8%BD%BD2.0%E7%89%88%E6%9C%AC.jpg)

2. Here, you can select Just Download.

Support the Arduino IDE

Since the release 1.x release in March 2015, the Arduino IDE has been downloaded **69,954,557** times — impressive! Help its development with a donation.

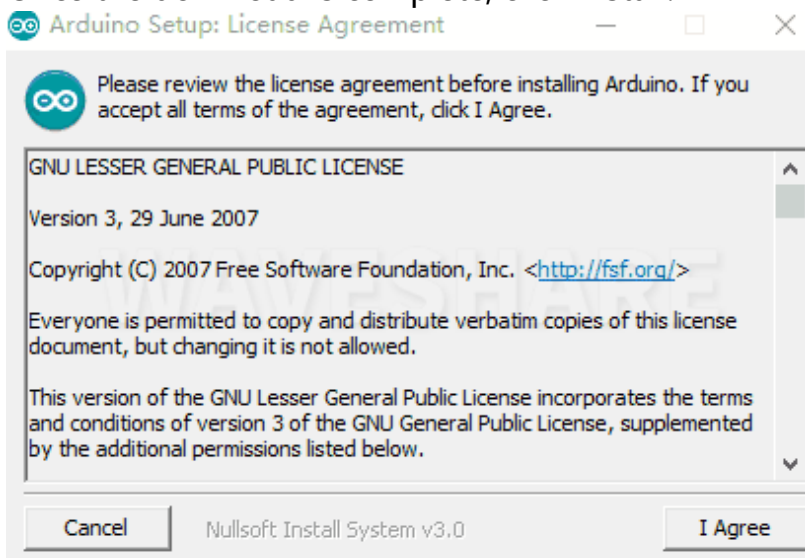
\$3	\$5	\$10	\$25	\$50	Other
-----	-----	------	------	------	-------



Learn more about [donating to Arduino](#).

(/wiki/File:%E4%BB%85%E4%B8%8B%E8%BD%BD%E4%B8%8D%E6%8D%90%E8%B5%A0.png)

3. Once the download is complete, click Install.



(/wiki/File:IDE%E5%AE%89%E8%A3%85%E6%B0%B4%E5%8D%B0-1.gif)

Notice: During the installation process, it will prompt you to install the driver, just click Install

Arduino IDE Interface

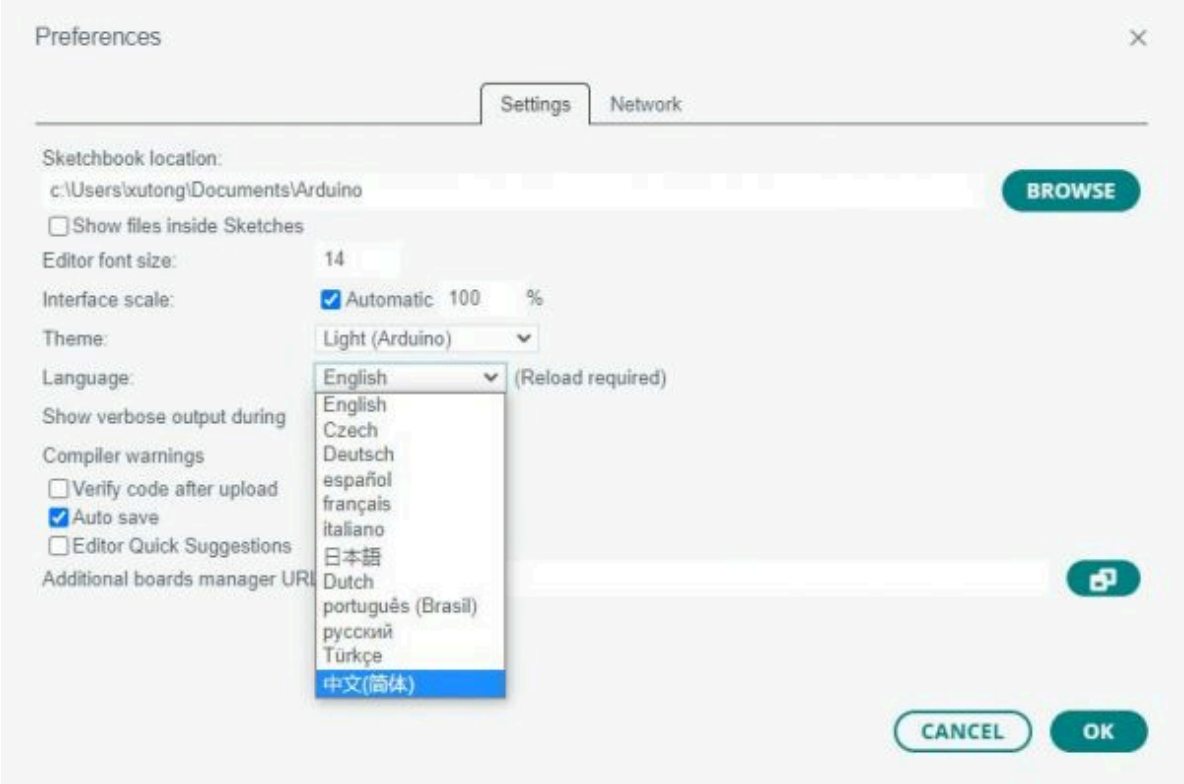
1. After the first installation, when you open the Arduino IDE, it will be in English. You can switch to other languages in File --> Preferences, or continue using the English interface.



(/wiki/File:%E9%A6%96%E9%80%89%E9%A1%B9-

%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87.jpg)

2. In the Language field, select the language you want to switch to, and click OK.



(/wiki/File:600px-%E9%A6%96%E9%80%89%E9%A1%B9-%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87ok.jpg)

Install Arduino-Pico Core in the Arduino IDE

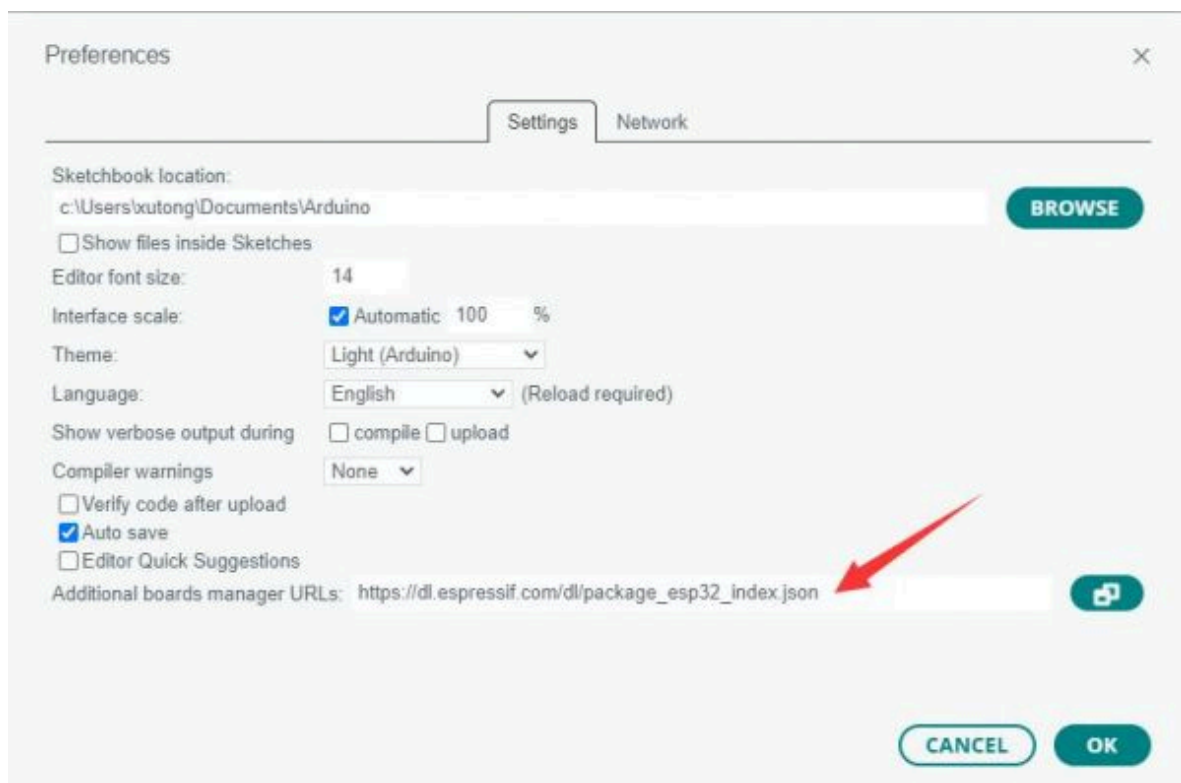
1. Open the Arduino IDE, click on the file in the top left corner, and select Preferences



(/wiki/File:RoArm-M1_Tutorial04.jpg)

2. Add the following link to the attached board manager URL, and then click OK

https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json

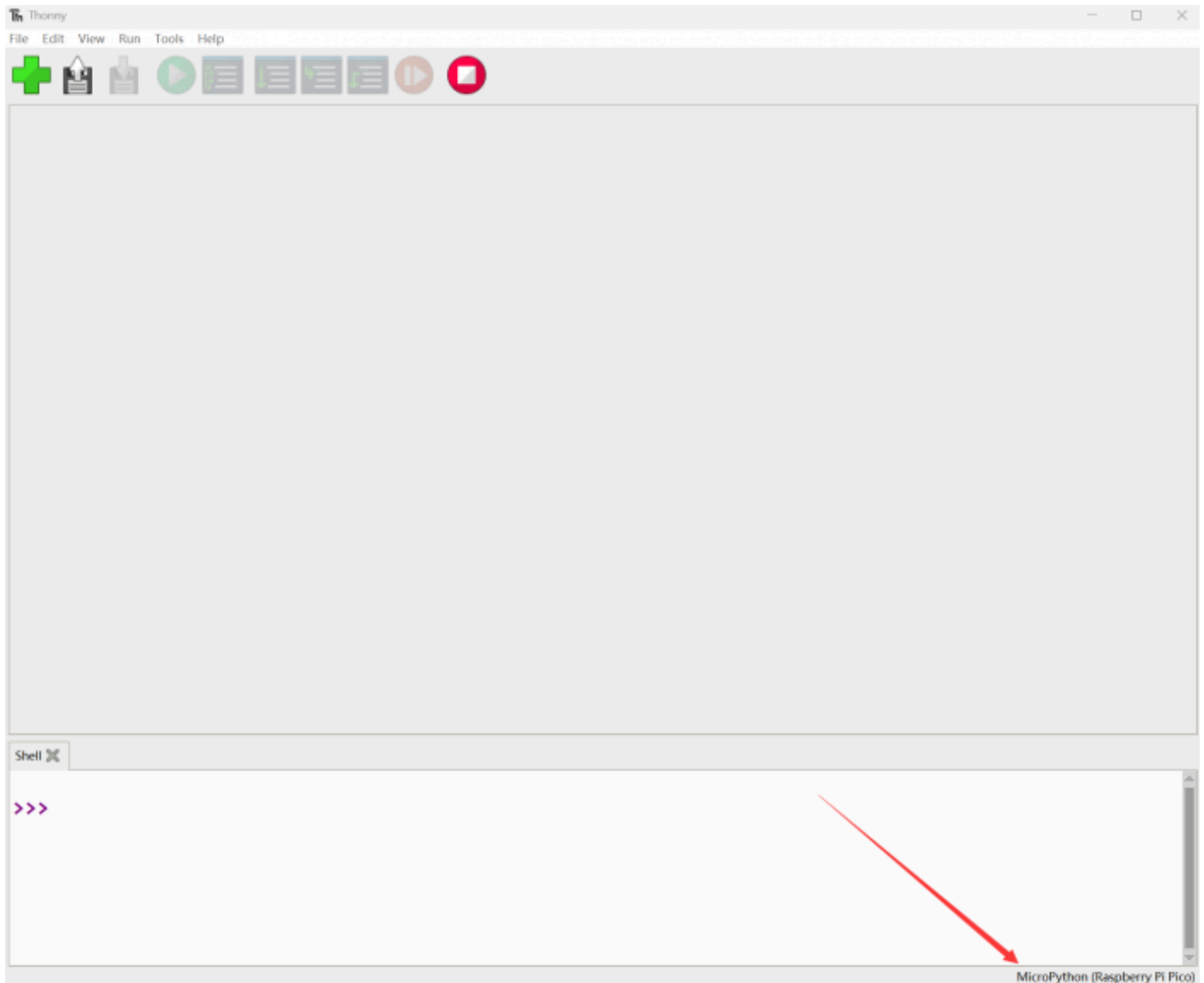


(/wiki/File:RoArm-M1_Tutorial_II05.jpg)

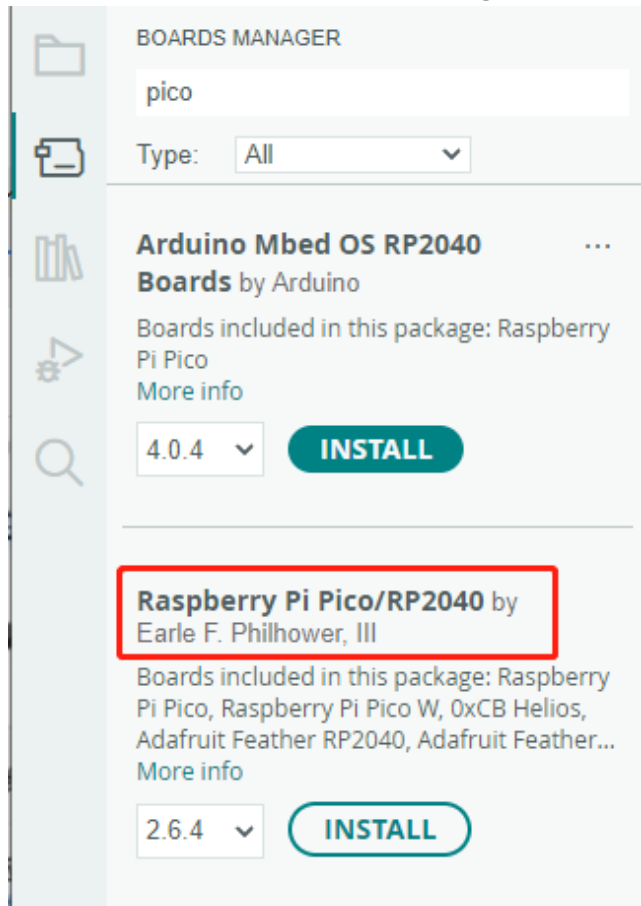
Note: If you already have an ESP32 board URL, you can use a comma to separate the URLs as follows:

https://dl.espressif.com/dl/package_esp32_index.json, https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json

3. Click Tools > Development Board > Board Manager > Search pico, as my computer has already been installed, it shows that it is installed



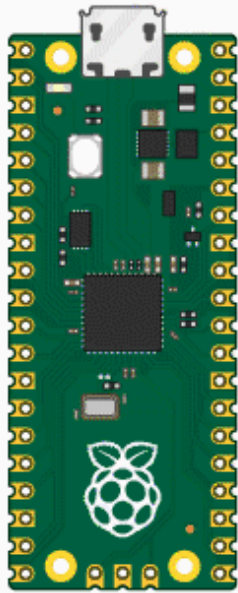
(/wiki/File:Pico_Get_Start_05.png)



(/wiki/File:Pico_Get_Start_06.png)

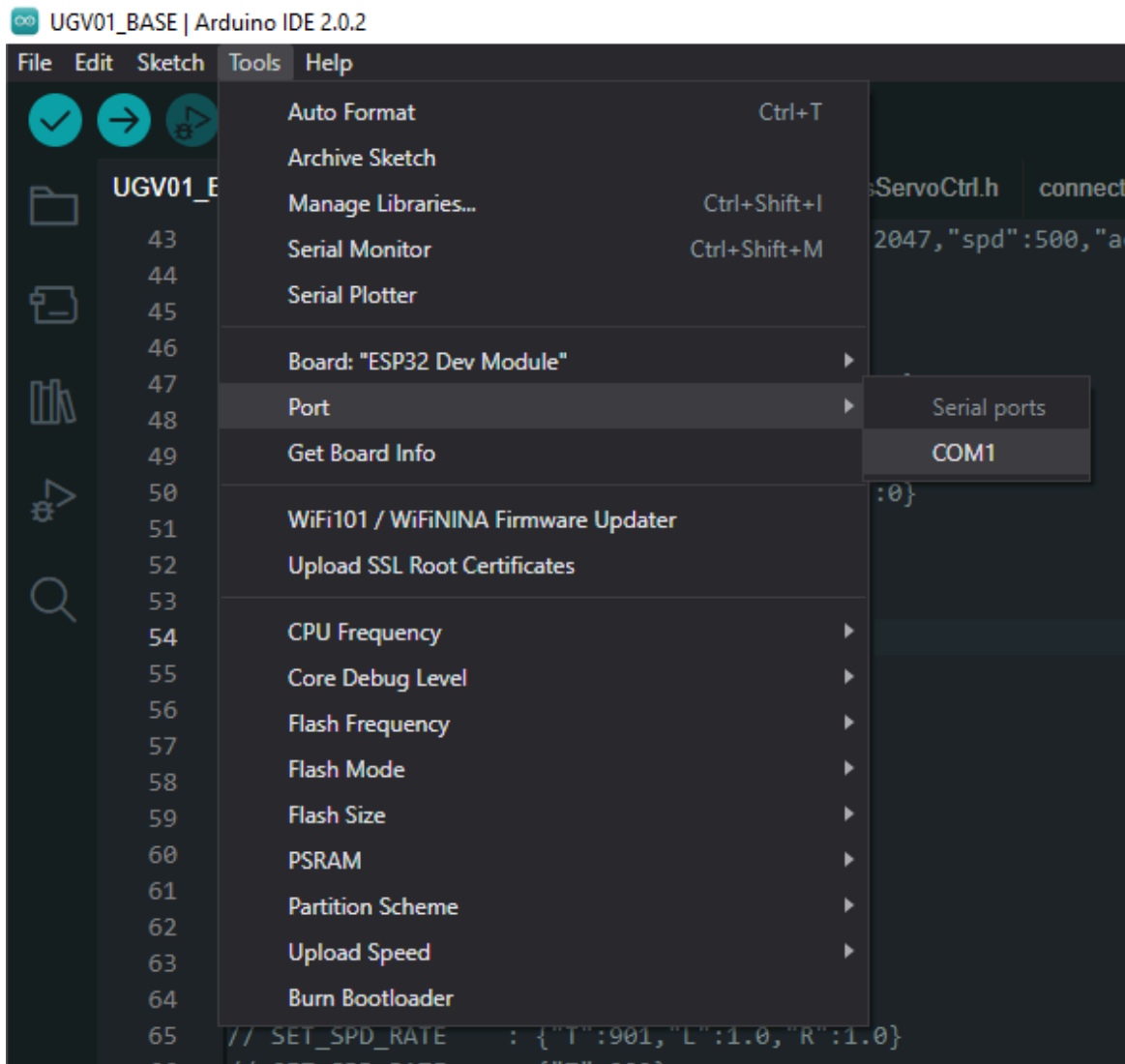
Upload Demo at the First Time

1. Press and hold the BOOTSET button on the Pico board, connect the pico to the USB port of the computer via the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).



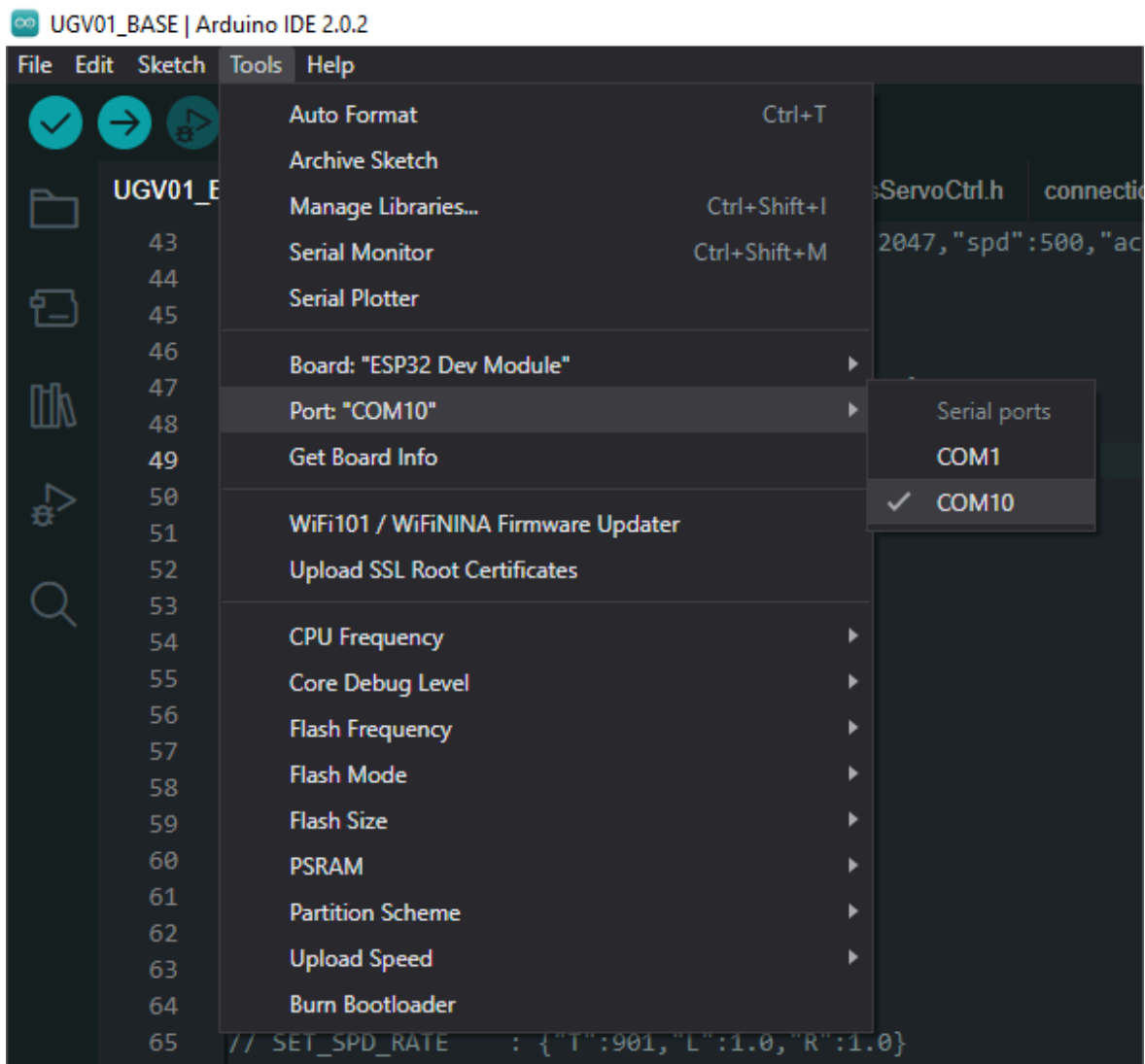
(/wiki/File:Pico_Get_Start.gif)

2. Download the program and open D1-LED.ino under the arduino\PWM\D1-LED path
3. Click Tools --> Port, remember the existing COM, do not click this COM (the COM displayed is different on different computers, remember the COM on your own computer)



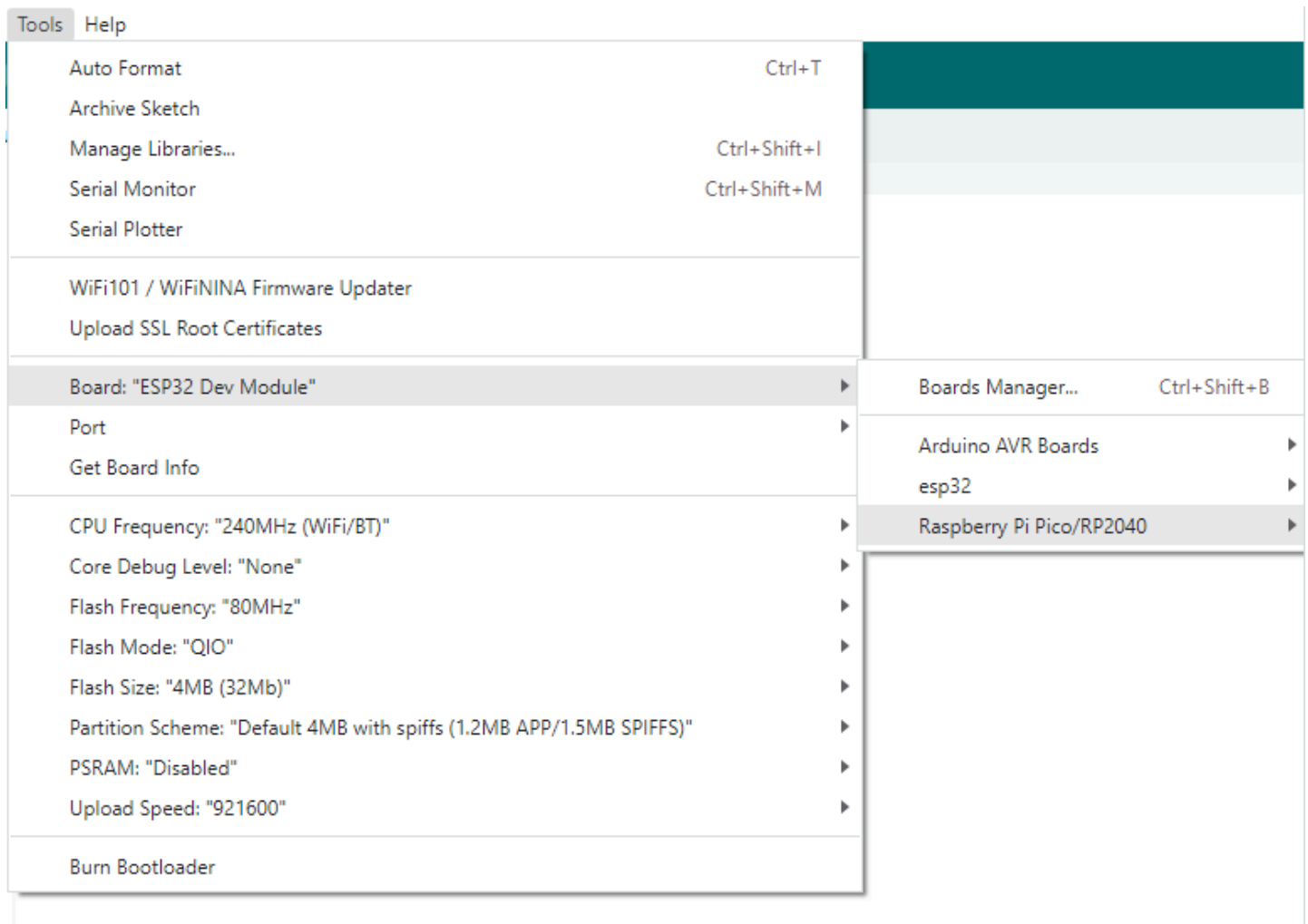
(/wiki/File:UGV1_download02EN.png)

4. Connect the driver board to the computer using a USB cable. Then, go to Tools > Port. For the first connection, select uf2 Board. After uploading, when you connect again, an additional COM port will appear



(/wiki/File:UGV1_download03EN.png)

5. Click Tools > Development Board > Raspberry Pi Pico > Raspberry Pi Pico or Raspberry Pi Pico 2



(/wiki/File:Pico_Get_Start02.png)

6. After setting it up, click the right arrow to upload the program



- If issues arise during this period, and if you need to reinstall or update the Arduino IDE version, it is necessary to uninstall the Arduino IDE completely. After uninstalling the software, you need to manually delete all contents within the C:\Users\[name]\AppData\Local\Arduino15 folder (you need to show hidden files to see this folder). Then, proceed with a fresh installation.

Open Source Demos

MircoPython video demo (github) (https://github.com/waveshareteam/Pico_MircoPython_Examples)

MicroPython firmware/Blink demos (C) (https://files.waveshare.com/wiki/common/Raspberry_Pi_Pico_Demo.zip)

Raspberry Pi official C/C++ demo (github) (<https://github.com/raspberrypi/pico-examples/>)

Raspberry Pi official micropython demo (github) (<https://github.com/raspberrypi/pico-micropython-examples>)

Arduino official C/C++ demo (github) (<https://github.com/earlephilhower/arduino-pico>)

FAQ

Question:When I power the Pico-Servo-Driver with the 4.5~26V screw connection, what is the right position of the jumper?

Answer:

Working voltage 5V (Pico) or 4.5~26V (VIN terminal).

You need to switch the USB power supply or battery power supply through the jumper cap, and you cannot use the USB and battery power supply at the same time. Either powered by USB or battery.

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.

Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

[Submit Now \(\)](#)